

Radix r^k FFTs: Matricial Representation and SDC/SDF Pipeline Implementation

Ainhoa Cortés, Igone Vélez, *Member, IEEE*, and Juan F. Sevillano, *Member, IEEE*

Abstract—This paper proposes to use the discrete Fourier transform (DFT) matrix factorization based on the Kronecker product to express the family of radix r^k single-path delay commutator/single-path delay feedback (SDC/SDF) pipeline fast Fourier transform (FFT) architectures. The matricial expressions of the radix r , r^2 , r^3 , and r^4 decimation-in-frequency (DIF) SDC/SDF pipeline architectures are derived. These expressions can be written using a small set of operators, resulting in a compact representation of the algorithms. The derived expressions are general in terms of r and the number of points of the FFT N . Expressions are given where it is not necessary that N is a power of r^k . The proposed set of operators can be mapped to equivalent hardware circuits. Thus, the designer can easily go from the matricial representations to their implementations and vice versa. As an example, the mapping of the operators is shown for radix 2, 2^2 , 2^3 , and 2^4 , and the details of the corresponding SDC/SDF pipeline FFT architectures are presented. Furthermore, a general expression is given for the SDC/SDF radix r^k pipeline architectures when $k > 4$. This general expression helps the designer to efficiently handle a wider design exploration space and select the optimum single-path architecture for a given value of N .

Index Terms—Discrete Fourier transform, fast Fourier transform, matrix decomposition, pipeline processing.

I. INTRODUCTION

THE fast Fourier transform (FFT) is an efficient algorithm to compute the discrete Fourier transform (DFT). The FFT is used in multiple applications where the frequency-domain representation of a signal has to be analyzed. In the communications area, the FFT has gained attention, because of its use in orthogonal frequency division multiplexing (OFDM) systems. Nowadays, the applications that use the FFT impose demanding specifications for the FFT processing, such as small silicon area, high throughput, short processing time, and reduced power consumption. For these applications, pipeline FFT architectures are the most appropriate. Two basic types of pipeline architectures can be found in the literature: delay commutator (DC) and delay feedback (DF). According to the number of lines of data used in these pipeline architectures, they can be further classified

into single-path (S) or multiple-path (M) architectures. Multiple-path architectures, such as [1]–[5], where data is input using several parallel paths, are used when the throughput needs to be increased for a given clock frequency of the FFT processor. In this paper, we focus on SDC/SDF pipeline architectures, where data is input to the FFT processor using a single path, one data after the other. A radix r SDC pipeline FFT algorithm was first proposed in [6] and a radix r SDF in [7].

The radix of the algorithm greatly influences the architecture of the FFT processor. A small radix is desirable, because it results in a simple butterfly. Nevertheless, a high radix reduces the number of twiddle factor multiplications. The idea of radix r^k algorithms is to try to achieve at the same time both a simple butterfly and a reduced number of twiddle factor multiplications. A 2^2 SDF pipeline architecture was first proposed in [8], a 2^3 SDF pipeline architecture in [9] and a 2^4 SDF pipeline architecture in [10]. These architectures are receiving much attention from the FFT pipeline hardware designers of OFDM receivers [9]–[14].

In [8]–[12] and [14] and, generally, in the related literature, the FFT pipeline algorithms are presented using summation based expressions or flow graph notations. On the one hand, the summation based expressions can provide a general representation of the algorithm in terms of the number of points of the FFT, N . However, the representation is not general in terms of the radix and it is hard for the designer to relate the summation based expressions to the final hardware implementation. On the other hand, the flow graphs are closer to the implementation. However, the FFT pipeline hardware designer has to draw different flow graphs for different lengths of the FFT and for different radices. Usually, both approaches limit their study to the case when the number of points of the FFT is $N = r^{kn}$. Furthermore, there is a lack of general expressions for radix r^k SDF pipeline architectures.

A different approach to represent FFT algorithms was first proposed in [15]. This approach relies on the Kronecker product (tensor product) to represent the matrix factorization of the DFT matrix. This matricial notation is employed to simplify the implementation of the FFT in a special purpose machine using parallel processing. A variation of the former approach is presented in [16]. In [17], the former works are extended to derive both previously defined and new FFT algorithms. The FFT matricial notation is used to provide general expressions in terms of N and the radix r . In addition, some of the hardware implications of the presented matricial expressions are explored. Later, [18] extends the matricial notation to other recursive algorithms and [19]–[21] relate this notation to its implementation in various computer architectures. Reference [22] suggests that the Kronecker product expression can also be identified with

Manuscript received May 28, 2008; accepted January 24, 2009. First published February 24, 2009; current version published June 17, 2009. The associate editor coordinating the review of this manuscript and approving it for publication was Prof. Jarmo Takala. A. Cortés holds the Torres Quevedo Grant PTQ05-02-02455, which was awarded by the Spanish Ministry of Education and Science, by the European Regional Development Fund and by the European Social Fund.

The authors are with the CEIT and TECNUN, University of Navarra, 20018 San Sebastián, Spain (e-mail: acortes@ceit.es; ivelez@ceit.es; jfsevillano@ceit.es).

Digital Object Identifier 10.1109/TSP.2009.2016276

specific VLSI implementation and in [23], the relationship between the Kronecker product and the hardware implementation is exploited to automatically generate radix r FFT architectures for field-programmable gate arrays (FPGAs). In [24] and [25], the matricial notation is used to describe some radix r pipeline architectures. However, the Kronecker product has been mainly used to optimize the implementation of recursive algorithms in computer architectures (e.g., [26]–[29]) and the matricial notation has not received much attention from the FFT pipeline hardware designers. To the authors best knowledge, the radix r^k algorithms have not been addressed using the matricial notation.

In this paper, the matricial representation is applied to radix r^k pipeline FFT architectures. Expressions for the radix r , r^2 , r^3 , and r^4 SDC and SDF pipeline architectures are derived. These expressions are general in terms of the radix r and the number of points of the FFT N , where N is a power of r , but not necessarily a power of r^k . The paper defines a set of operators, so that the designer can easily relate the matricial representations to their hardware implementations. The hardware implementations are detailed for $r^k = 2^k$, with $k = 1, 2, 3, 4$, and N any power of r . The radix r^2 , r^3 , and r^4 SDC are novel architectures. It is shown that the same matricial expression leads to both the SDC and the SDF pipeline architectures. Furthermore, a general expression is provided for r^k SDC/SDF pipeline algorithms when N is a power of r , but not necessarily a power of r^k . These new architectures open new possible choices for the designer. The matricial notation helps the designer to efficiently handle the increased design exploration space and select the most appropriate single-path pipeline architecture for an application. Although the paper only presents results for decimation-in-frequency (DIF) algorithms, the decimation-in-time (DIT) algorithms can be obtained by matricial transposition.

II. REVIEW OF THE DFT MATRIX FACTORIZATION

In this section, the factorization method of the DFT matrix proposed in [17] is reviewed. The DFT can be expressed matricially as

$$\text{DFT}(\mathbf{x}) = \mathbf{T}_N \cdot \mathbf{x} \quad (1)$$

where $\mathbf{x}^T = \{x_0 \ x_1 \ x_2 \ \dots \ x_{N-1}\}$ and $\mathbf{T}_N$ is the size N square matrix

$$(\mathbf{T}_N)_{mn} = e^{-j\frac{mn^2\pi}{N}}. \quad (2)$$

In order to factorize $\mathbf{T}_N$, [17] defines

$$\mathbf{T}_N = \mathbf{P}_N^{(r)} \cdot \mathbf{T}_N^{(p)} \quad (3)$$

where r is the radix of the algorithm, $p = N/r$ is a positive integer ($p \in \mathbb{N}^*$) and $\mathbf{P}_N^{(r)}$ is the stride permutation matrix. This matrix, $\mathbf{P}_N^{(r)}$, is defined by its effect on a vector

$$\mathbf{P}_N^{(r)} \cdot \{x_0 \ x_1 \ \dots \ x_{N-1}\}^T = \{y_0 \ y_1 \ \dots \ y_{p-1}\}^T \quad (4)$$

with $\mathbf{y}_i = \{x_i \ x_{i+p} \ x_{i+2p} \ \dots \ x_{i+(r-1)p}\}$. The matrix $\mathbf{T}_N^{(p)}$ can be expressed as

$$\mathbf{T}_N^{(p)} = [\mathbf{I}_r \otimes \mathbf{T}_{N/r}] \cdot \mathbf{D}_N^{(r)} \cdot [\mathbf{T}_r \otimes \mathbf{I}_{N/r}] \quad (5)$$

where $\mathbf{I}_z$ is the identity matrix of size z . The symbol $\otimes$ represents the Kronecker product. Given an $m \times n$ matrix $\mathbf{A}$ and a matrix $\mathbf{B}$, the Kronecker product is defined as

$$\mathbf{A} \otimes \mathbf{B} = \begin{bmatrix} a_{0,0}\mathbf{B} & a_{0,1}\mathbf{B} & \dots & a_{0,n-1}\mathbf{B} \\ a_{1,0}\mathbf{B} & a_{1,1}\mathbf{B} & \dots & a_{1,n-1}\mathbf{B} \\ \vdots & \vdots & & \vdots \\ a_{m-1,0}\mathbf{B} & a_{m-1,1}\mathbf{B} & \dots & a_{m-1,n-1}\mathbf{B} \end{bmatrix}. \quad (6)$$

$\mathbf{D}_N^{(r)}$ is a diagonal matrix given by

$$\mathbf{D}_N^{(r)} = \text{quasidiag}(\{\mathbf{I}_p \ \mathbf{K}_p^1 \ \mathbf{K}_p^2 \ \dots \ \mathbf{K}_p^{r-1}\}) \quad (7)$$

with $\mathbf{K}_p = \text{diag}(\{1 \ e^{-j\frac{2\pi}{N}} \ e^{-j\frac{2^2\pi}{N}} \ \dots \ e^{-j\frac{(p-1)^2\pi}{N}}\})$.

Replacing (5) in (3), we arrive to

$$\mathbf{T}_N = \mathbf{P}_N^{(r)} \cdot [\mathbf{I}_r \otimes \mathbf{T}_{N/r}] \cdot \mathbf{D}_N^{(r)} \cdot [\mathbf{T}_r \otimes \mathbf{I}_{N/r}]. \quad (8)$$

Equation (8) allows us to write a DFT matrix in terms of smaller DFT matrices. This process can be repeated recursively until $\mathbf{T}_N$ is expressed in terms of $\mathbf{T}_r$. Equation (8) is a matricial representation of the recursion used in the well-known Cooley-Tukey FFT [30].

III. MAPPING THE DFT MATRIX TO A RADIX r DIF PIPELINE FFT ARCHITECTURE

In Sections III–VI, we propose a unified matricial representation for the radix r , r^2 , r^3 and r^4 SDC and SDF architectures. It will be shown that the same matricial representation is mapped to both the SDC and SDF architectures. Section VII presents a general expression for r^k SDC/SDF architectures.

A. Matrix Representation

Applying the properties defined in Appendix A to (8), the following expression can be obtained

$$\mathbf{T}_N = \mathbf{Q}_N \cdot \mathbf{T}_N^{(p_1)} \quad (9a)$$

$$\begin{aligned} \mathbf{T}_N^{(p_1)} &= \mathbf{H}^{(n_1-1)} \cdot \mathbf{H}^{(n_1-2)} \dots \mathbf{H}^{(0)} \\ &= \prod_{m=0}^{n_1-1} \mathbf{H}^{(n_1-m-1)} \end{aligned} \quad (9b)$$

$$\begin{aligned} \mathbf{H}^{(i)} &= [\mathbf{I}_{r^i} \otimes \mathbf{D}_{N/r^i}^{(r)}] \cdot [\mathbf{I}_{r^i} \otimes (\mathbf{P}_{N/r^i}^{(r)})^{-1}] \\ &\quad \cdot [\mathbf{I}_{N/r} \otimes \mathbf{T}_r] \cdot [\mathbf{I}_{r^i} \otimes \mathbf{P}_{N/r^i}^{(r)}] \end{aligned} \quad (9c)$$

$$\mathbf{Q}_N = \prod_{i=0}^{n_1-1} [\mathbf{I}_{r^i} \otimes \mathbf{P}_{N/r^i}^{(r)}] \quad (9d)$$

where $N = r^{n_1}$ and $p_1 = r^{n_1-1}$, with $\{n_1, p_1\} \in \mathbb{N}^*$. This expression is derived in Appendix B. Equations (9a)–(9d) represent a radix r DIF FFT algorithm.

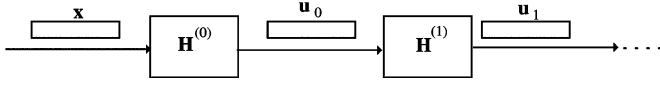
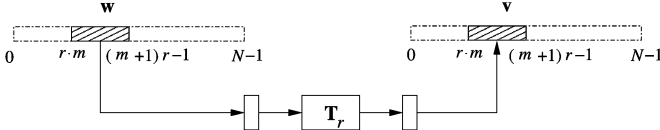


Fig. 1. Pipeline implementation of FFT stages defined by (9b).

Fig. 2. Sequential implementation of Kronecker product $\mathbf{v} = [\mathbf{I}_{N/r} \otimes \mathbf{T}_r]\mathbf{w}$.

B. Mapping to an SDC Pipeline Architecture

Equation (9a) means that the output vector of the operations carried out in (9b) should be reordered according to the shuffling matrix $\mathbf{Q}_N$ in (9d). $\mathbf{Q}_N$ corresponds to a bit- r reversed ordering. This final reordering is typical of DIF FFT algorithms. This work considers that this reordering is not implemented within the FFT processor, as it is common in the related literature (e.g., [3]–[5], [10]–[12]).

The product of (9b) explicitly represents the stages of the FFT algorithm. The output of stage 0 is a new vector $\mathbf{u}_0 = \mathbf{H}^{(0)} \cdot \mathbf{x}$. The hardware in stage 0 must process the input vector $\mathbf{x}$ to generate $\mathbf{u}_0$. The output of stage 1 is $\mathbf{u}_1 = \mathbf{H}^{(1)} \cdot \mathbf{u}_0$; i.e., stage 1 must process the output of stage 0 to generate $\mathbf{u}_1$, and so on. These sequence of operations can be efficiently implemented in a pipeline architecture as shown in Fig. 1. There are as many processing blocks as stages of the FFT algorithm. The data vector to be processed is input to each processing block $\mathbf{H}^{(i)}$ sequentially, one element after the other, using a single data-path. These processing blocks also output the resulting vector sequentially using a single data-path.

The processing defined by (9c) for each stage of the FFT can be divided in the following four steps.

- Step 1: Shuffle the stage input data according to $\mathbf{I}_{r^i} \otimes \mathbf{P}_{N/r^i}^{(r)}$.
- Step 2: Perform the butterfly operations defined by $[\mathbf{I}_{N/r} \otimes \mathbf{T}_r]$ over the shuffled data.
- Step 3: Shuffle the data resulting from the butterfly operations according to $\mathbf{I}_{r^i} \otimes (\mathbf{P}_{N/r^i}^{(r)})^{-1}$. This step undoes the data shuffling carried out in Step 1.
- Step 4: Multiply the unshuffled data by the twiddle factors defined by $\mathbf{I}_{r^i} \otimes \mathbf{D}_{N/r^i}^{(r)}$.

The most straightforward implementation of the i th stage is an SDC pipeline architecture. The term $[\mathbf{I}_{N/r} \otimes \mathbf{T}_r]$ in Step 2 means that we have to take each nonoverlapping size r subvector of the shuffled data, and apply the operations defined by $\mathbf{T}_r$ to produce each nonoverlapping size r subvector of the output vector. This step can be implemented using a single hardware processing element that performs the arithmetic operations defined by $\mathbf{T}_r$. This processing element has r inputs, one per element of the subvector, and produces its result as r parallel outputs. The implementation of the Kronecker product in Step 2 is illustrated in Fig. 2, where the m th iteration is depicted.

As Step 2 requires a parallel input, Step 1 has to present the subvectors to $\mathbf{T}_r$ in parallel. The shuffling defined by $\mathbf{P}_{N/r^i}^{(r)}$ plus the serial to parallel data transformation can be implemented by a commutator that distributes the data in r lines, where $r-1$ of the data lines are delay lines of length N/r^{i+1} . The commutator switches every N/r^{i+1} data. The Kronecker product $\mathbf{I}_{r^i} \otimes \mathbf{P}_{N/r^i}^{(r)}$ means that only the set of commutator and delay lines corresponding to $\mathbf{P}_{N/r^i}^{(r)}$ needs to be implemented in Step 1. In order to process the whole input vector, it must operate r^i times.

Step 3 must undo the shuffling carried out in Step 1. It receives r parallel input data and will output the data serially in a single data-path. The unshuffling defined by $(\mathbf{P}_{N/r^i}^{(r)})^{-1}$ plus the parallel to serial data transformation can be implemented using $r-1$ delay lines of length N/r^{i+1} and a commutator that selects which of the r lines is connected to the output. The commutator switches also every N/r^{i+1} data. Again, the Kronecker product $\mathbf{I}_{r^i} \otimes (\mathbf{P}_{N/r^i}^{(r)})^{-1}$ means that only the set of delay lines and commutator corresponding to $(\mathbf{P}_{N/r^i}^{(r)})^{-1}$ needs to be implemented.

Finally, matrix $\mathbf{D}_{N/r^i}^{(r)}$ of Step 4 is a diagonal matrix. Therefore, we have to multiply each element of the input vector by each element of the diagonal of $\mathbf{D}_{N/r^i}^{(r)}$. This step can be implemented using only one complex multiplier and a lookup table (LUT) of size N/r^i . The twiddle factors are read sequentially from the LUT. The Kronecker product $\mathbf{I}_{r^i} \otimes \mathbf{D}_{N/r^i}^{(r)}$ means that the processing must be repeated r^i times to process the whole input vector.

Fig. 3(a) presents the implementation of the i th stage when $r = 2$. The pipeline architectures such as the one in Fig. 3(a) achieve a high throughput: the first input data for the next FFT can be input just after the last input data of the current FFT.

In order to simplify the notation and clarify the relationship between the matricial representation and the hardware implementation, several operators are defined in Appendix C. Using these operators, equations (9a)–(9d) can be rewritten as

$$\mathbf{T}_N = \mathbf{Q}_N \cdot \mathbf{T}_N^{(p_1)} \quad (10a)$$

$$\mathbf{T}_N^{(p_1)} = \prod_{m=0}^{n_1-1} \mathbf{H}^{(n_1-m-1)} \quad (10b)$$

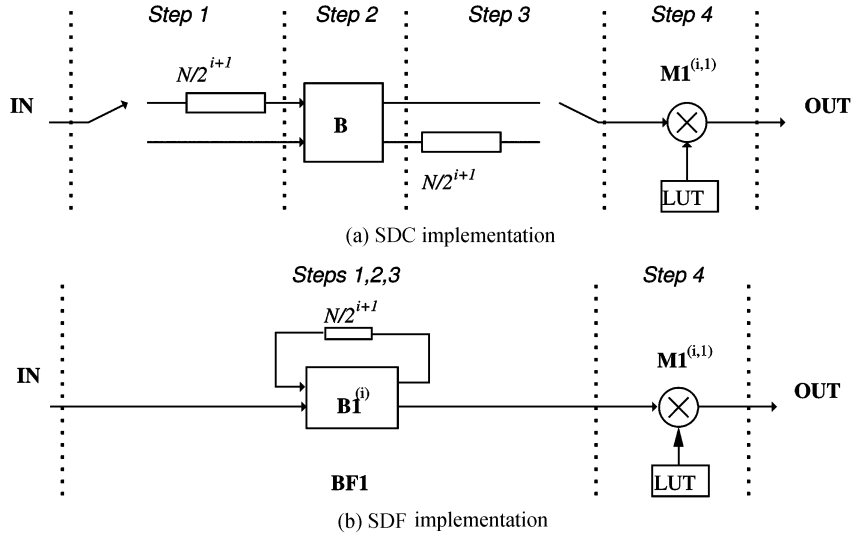
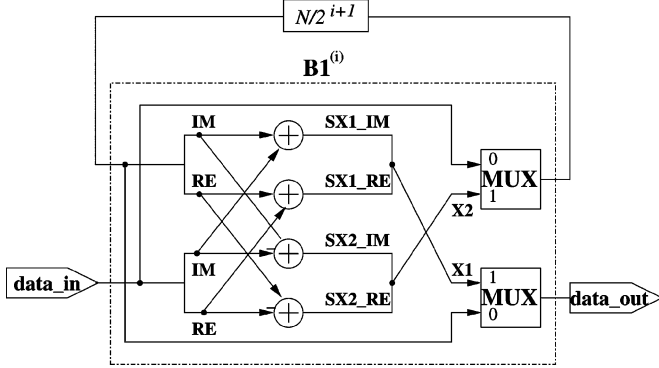
$$\mathbf{H}^{(i)} = \mathbf{M}\mathbf{1}^{(i,1)} \cdot (\mathbf{S}^{(i)})^{-1} \cdot \mathbf{B} \cdot \mathbf{S}^{(i)} \quad (10c)$$

$$\mathbf{Q}_N = \prod_{i=0}^{n_1-1} \mathbf{S}^{(i)} \quad (10d)$$

where it must be noticed that $\mathbf{M}\mathbf{1}^{(i,1)} = \mathbf{I}_{r^i} \otimes \mathbf{D}_{N/r^i}^{(r)}$, because $\mathbf{P}_r^{(r)} = \mathbf{I}_r$. Operators $\mathbf{S}^{(i)}$ and $(\mathbf{S}^{(i)})^{-1}$ define the shuffling in each stage, and operators $\mathbf{B}$ and $\mathbf{M}\mathbf{1}^{(i,1)}$ define the arithmetic operations.

C. Mapping to an SDF Pipeline Architecture

As the delay lines used to implement the unshuffling $(\mathbf{S}^{(i)})^{-1}$ are filled at the same time as the delay lines used to implement the shuffling $\mathbf{S}^{(i)}$ are emptied, this delay line hardware can be shared. Thus, a more efficient implementation can be achieved if Steps 1 to 3 are combined into a single device $\mathbf{B}\mathbf{F}\mathbf{1}^{(i)} = \mathbf{S}^{(i)} \cdot \mathbf{B} \cdot (\mathbf{S}^{(i)})^{-1}$. This device implements the hardware needed

Fig. 3. Diagram of the i th stage of the radix $r = 2$ DIF pipeline architectures.Fig. 4. Architecture of $\mathbf{BF1}^{(i)}$ for an SDF FFT when $r = 2$.

to perform the arithmetic operations given by $\mathbf{B}$ and $r - 1$ delay lines of length N/r^{i+1} and a set of commutators to perform the shuffling. The resulting implementation is an SDF architecture. Fig. 3(b) shows the implementation of a stage when $r = 2$. The implementation of module $\mathbf{BF1}^{(i)}$ when $r = 2$ is detailed in Fig. 4. The shuffling is achieved using a single delay feedback line of length $N/2^{i+1}$, and two multiplexers that are switched every $N/2^{i+1}$ data [14]. If we compare the i th stages

of SDC architectures and SDF architectures, it can be seen that the memory requirements are halved.

IV. MAPPING THE DFT MATRIX TO A RADIX r^2 DIF PIPELINE FFT ARCHITECTURE

A. Matrix Representation

The matricial representation of the radix r^2 DIF SDC/SDF architectures is derived in Appendix D. Using the operators defined in Appendix C, the matricial representation can be written as shown in equations (11a)–(11d) at the bottom of the page, where $p_2 = r^{2(n_2-1)}$, with $\{n_2, p_2\} \in \mathbb{N}^*$. In order to link the above expression to Appendix D, note that when $N = r^{2n_2+1}$, $\mathbf{S}^{(2n_2)} = \mathbf{I}_{r^{2n_2}} \otimes \mathbf{P}_r^{(r)}$, where $\mathbf{P}_r^{(r)} = \mathbf{I}_r$.

B. Mapping to an SDC Pipeline Architecture

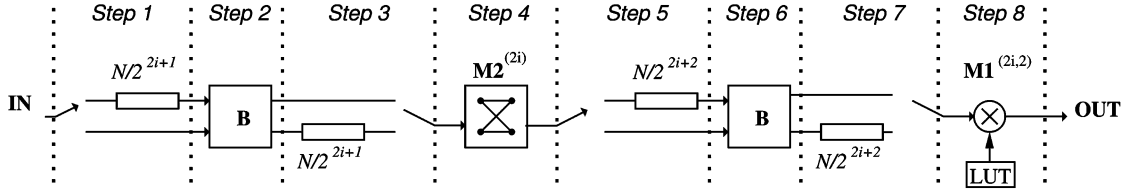
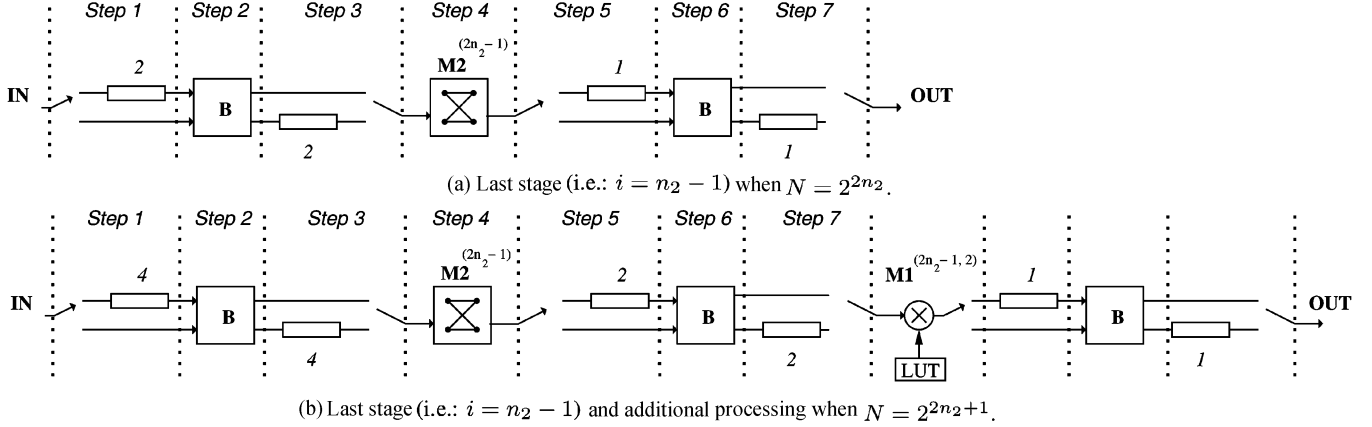
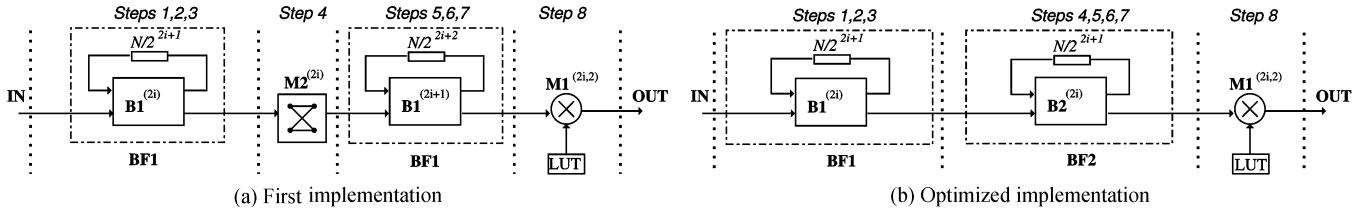
Equation (11c) shows that each stage of the FFT can be divided in eight steps. Step 1 corresponds to the shuffling defined by $\mathbf{S}^{(2i)}$ and Step 8 to the multiplication by the twiddle factors given by $\mathbf{M1}^{(2i,2)}$. These steps are similar to those in Section III-B, except for Step 4. The input vector to Step 4 is multiplied by the twiddle factors defined by the new arithmetic

$$\mathbf{T}_N = \begin{cases} \mathbf{Q}_N \cdot \mathbf{T}_N^{(p_2)}, & \text{for } N = r^{2n_2} \\ \mathbf{Q}_N \cdot (\mathbf{S}^{(2n_2)})^{-1} \cdot \mathbf{B} \cdot \mathbf{S}^{(2n_2)} \cdot \mathbf{T}_N^{(p_2)}, & \text{for } N = r^{2n_2+1} \end{cases} \quad (11a)$$

$$\mathbf{T}_N^{(p_2)} = \prod_{m=0}^{n_2-1} \mathbf{H}^{(n_2-m-1)} \quad (11b)$$

$$\mathbf{H}^{(i)} = \mathbf{M1}^{(2i,2)} \cdot (\mathbf{S}^{(2i+1)})^{-1} \cdot \mathbf{B} \cdot \mathbf{S}^{(2i+1)} \cdot \mathbf{M2}^{(2i)} \cdot (\mathbf{S}^{(2i)})^{-1} \cdot \mathbf{B} \cdot \mathbf{S}^{(2i)} \quad (11c)$$

$$\mathbf{Q}_N = \prod_{i=0}^{n_1-1} \mathbf{S}^{(i)} \quad (11d)$$

Fig. 5. Diagram of the i th stage of the radix $r^2 = 2^2$ DIF SDC pipeline architecture.Fig. 6. Differences in the implementation of the SDC DIF algorithm for $r = 2^2$ when $N = 2^{2n_2}$ and $N = 2^{2n_2+1}$.Fig. 7. Diagrams of the i th stage of the radix $r^2 = 2^2$ DIF SDF pipeline architecture.

operator $\mathbf{M2}^{(2i)}$, which is a diagonal matrix. Thus, we have to multiply each element of the input vector by each element of the diagonal of $\mathbf{M2}^{(2i)}$. From (36), we easily obtain that this step can be implemented using only one complex multiplier and a LUT of size r^2 from where the twiddle factors are read sequentially. Each twiddle factor is used N/r^{2i+2} times before reading the next value of the LUT.

Fig. 5 presents the i th stage of the SDC implementation when $r^2 = 2^2$. The delay commutator structure of Step 1 has a delay line of size $N/2^{2i+1}$, and outputs the subvector required as input to the butterfly of Step 2. The delay commutator structure of Step 3 has also one delay line of length $N/2^{2i+1}$. The multiplications of Step 4 can be efficiently implemented when $r^2 = 2^2$. In this case, (36) is

$$\mathbf{M2}^{(a)} = \mathbf{I}_{2^a} \otimes \mathbf{D}_{2^2}^{(2)} \otimes \mathbf{I}_{N/2^{a+2}} \quad (12)$$

where $\mathbf{D}_{2^2}^{(2)} = \text{diag}(\{1 \ 1 \ 1 \ -j\})$. Thus, the complex multiplier and the LUT can be replaced by a simple swap of the real and imaginary parts of its input data. Steps 5 and 7 can be implemented using delay commutator structures with delays of length $N/2^{2i+2}$.

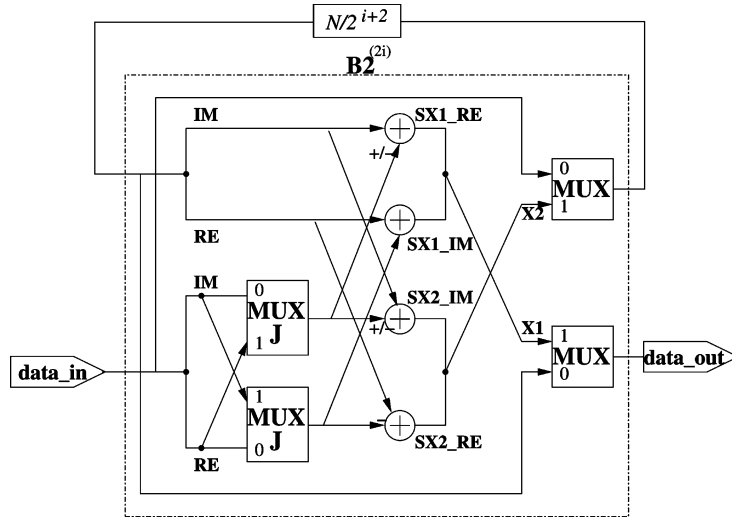
Fig. 6(a) shows the last stage when $N = 2^{2n_2}$. In general, when $N = r^{2n_2}$, no complex multiplier is needed to implement

the term $\mathbf{M1}$ of $\mathbf{H}^{(n_2-1)}$, because $\mathbf{D}_{N/r^{2(n_2-1)}}^{(r^2)} = \mathbf{D}_{r^2}^{(r^2)} = \mathbf{I}_{r^2}$ and then $\mathbf{M1}^{(2(n_2-1),2)} = \mathbf{I}_N$. However, when $N = r^{2n_2+1}$, a complex multiplier is needed to implement the term $\mathbf{M1}$ of $\mathbf{H}^{(n_2-1)}$, as shown for example in Fig. 6(b). In addition, (11a) indicates that when $N = r^{2n_2+1}$ an additional processing is required.

C. Mapping to an SDF Pipeline Architecture

A more efficient implementation can be achieved combining Steps 1 to 3 and Steps 5 to 7 into devices $\mathbf{BF1}^{(2i)}$ and $\mathbf{BF1}^{(2i+1)}$, respectively. A diagram of the i th stage of the resulting SDF architecture when $r^2 = 2^2$ is shown in Fig. 7(a).

This architecture can be further optimized if we combine more elements by defining a new device $\mathbf{BF2}^{(a)} = \mathbf{BF1}^{(a+1)} \cdot \mathbf{M2}^{(a)}$. Fig. 8 details the architecture of $\mathbf{BF2}^{(2i)}$ when $r^2 = 2^2$. This device exploits the fact that no actual multiplications are needed to implement $\mathbf{M2}^{(2i)}$. The structure and operation of $\mathbf{BF2}^{(2i)}$ is similar to that of $\mathbf{BF1}^{(2i+1)}$, except for MUX J. The multiplexors MUX J are used to handle efficiently the multiplications by $-j$ defined by $\mathbf{M2}^{(2i)}$. They are implemented by swapping the real and imaginary parts and changing the sign of the real part of the input [14]. The diagram of the i th stage of the resulting SDF implementation can be seen in Fig. 7(b).

Fig. 8. Architecture of $\text{BF}2^{(2i)}$ for an SDF FFT when $r^2 = 2^2$.

V. MAPPING THE DFT MATRIX TO A RADIX r^3 DIF PIPELINE FFT ARCHITECTURE

A. Matrix Representation

The matricial representation of the radix r^3 DIF SDC/SDF architectures is derived in Appendix E, and can be written as shown in equations (13a)–(13d), which can be seen at the bottom of the page, where $p_3 = r^{3(n_3-1)}$, with $\{n_3, p_3\} \in \mathbb{N}^*$. In order to link the above expression to Appendix E, note that when $N = r^{3n_3+1}$, $\mathbf{S}^{(3n_3)} = \mathbf{I}_{r^{3n_3}} \otimes \mathbf{P}_r^{(r)} = \mathbf{I}_N$. When $N = r^{3n_3+2}$, $\mathbf{S}^{(3n_3+1)} = \mathbf{I}_{r^{3n_3+1}} \otimes \mathbf{P}_r^{(r)} = \mathbf{I}_N$.

B. Mapping to an SDC Pipeline Architecture

Equation (13c) shows that each stage of the FFT can be divided in twelve steps, each step corresponding to an operator. Step 1 corresponds to the shuffling according to $\mathbf{S}^{(3i)}$ and Step 12 to the multiplication by the twiddle factors given by the diagonal matrix $\mathbf{M}1^{(3i,3)}$. The steps are similar to those in Sections III-B and IV-B, except for Step 8. In this step, the output data of Step 7 are multiplied by the twiddle factors as defined by the new arithmetic operator $\mathbf{M}3^{(3i,3)}$. This operator is a diagonal matrix, and thus, it can be implemented using

only one complex multiplier and a LUT from where the twiddle factors are read sequentially. In this case, from (37), we easily conclude that a LUT of size r^3 is enough. Each twiddle factor is used N/r^{3i+3} times before proceeding to read the next value from the LUT.

When $r^3 = 2^3$, the twiddle factors given by (37) are

$$\mathbf{D}_{2^3}^{(2^2)} = \text{diag}(\{ \begin{matrix} 1 & 1 & 1 & (c(\frac{\pi}{4}) - js(\frac{\pi}{4})) \\ 1 & -j & 1 & (-c(\frac{\pi}{4}) - js(\frac{\pi}{4})) \end{matrix} \}) \quad (14)$$

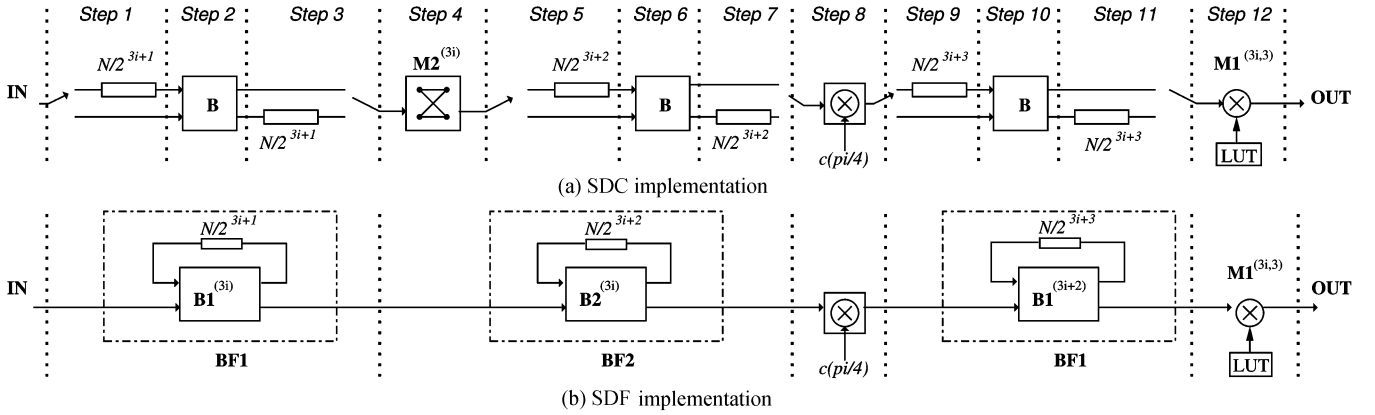
where $c(\cdot) = \cos(\cdot)$ and $s(\cdot) = \sin(\cdot)$. Therefore, the complex multiplier and LUT of Step 8 can be avoided. All the operations required by $\mathbf{M}3^{(a,3)}$ can be implemented in a device able to swap the real and imaginary parts of the input data and perform two real multiplications by the constant value $\cos(\pi/4) = \sin(\pi/4) = \sqrt{2}/2$ and a complex addition. The implementation of the i th stage of the DIF SDC architecture when $r^3 = 2^3$ can be seen in Fig. 9(a). The final processing depends on the length of the FFT. When $N = r^{3n_3}$, no complex multiplier is needed to perform the twiddle factor multiplications given by $\mathbf{M}1^{(3(n_3-1),3)}$ of $\mathbf{H}^{(n_3-1)}$. Nevertheless, when $N = r^{3n_3+1}$ or $N = r^{3n_3+2}$, a complex multiplier is needed to implement $\mathbf{M}1^{(3(n_3-1),3)}$ in $\mathbf{H}^{(n_3-1)}$. In addition, (13a) indicates that further processing is needed.

$$\mathbf{T}_N = \begin{cases} \mathbf{Q}_N \cdot \mathbf{T}_N^{(p_3)}, & \text{for } N = r^{3n_3} \\ \mathbf{Q}_N \cdot (\mathbf{S}^{(3n_3)})^{-1} \cdot \mathbf{B} \cdot \mathbf{S}^{(3n_3)} \cdot \mathbf{T}_N^{(p_3)}, & \text{for } N = r^{3n_3+1} \\ \mathbf{Q}_N \cdot (\mathbf{S}^{(3n_3+1)})^{-1} \cdot \mathbf{B} \cdot \mathbf{S}^{(3n_3+1)} \cdot \mathbf{M}2^{(3n_3)} \cdot (\mathbf{S}^{(3n_3)})^{-1} \cdot \mathbf{B} \cdot \mathbf{S}^{(3n_3)} \cdot \mathbf{T}_N^{(p_3)}, & \text{for } N = r^{3n_3+2} \end{cases} \quad (13a)$$

$$\mathbf{T}_N^{(p_3)} = \prod_{m=0}^{n_3-1} \mathbf{H}^{(n_3-m-1)} \quad (13b)$$

$$\mathbf{H}^{(i)} = \mathbf{M}1^{(3i,3)} \cdot (\mathbf{S}^{(3i+2)})^{-1} \cdot \mathbf{B} \cdot \mathbf{S}^{(3i+2)} \cdot \mathbf{M}3^{(3i,3)} \cdot (\mathbf{S}^{(3i+1)})^{-1} \cdot \mathbf{B} \cdot \mathbf{S}^{(3i+1)} \cdot \mathbf{M}2^{(3i)} \cdot (\mathbf{S}^{(3i)})^{-1} \cdot \mathbf{B} \cdot \mathbf{S}^{(3i)} \quad (13c)$$

$$\mathbf{Q}_N = \prod_{i=0}^{n_1-1} \mathbf{S}^{(i)} \quad (13d)$$

Fig. 9. Diagrams of the i th stage of radix $r^3 = 2^3$ DIF pipeline architectures.

C. Mapping to an SDF Pipeline Architecture

Steps 1 to 3 and 9 to 11 can be combined into devices $\mathbf{BF1}^{(3i)}$ and $\mathbf{BF1}^{(3i+2)}$, respectively. Steps 4 to 7 can be combined into a device $\mathbf{BF2}^{(3i)}$. This way a more efficient SDF architecture is obtained. Fig. 9(b) presents a diagram of the implementation of a stage of the SDF architecture when $r^3 = 2^3$.

VI. MAPPING THE DFT MATRIX TO A RADIX r^4 DIF PIPELINE FFT ARCHITECTURE

A. Matrix Representation

The matricial representation of the radix r^4 DIF SDC/SDF architectures is derived in Appendix F and can be written as (15a)–(15d), shown at the bottom of the page, where $p_4 =$

$r^{4(n_4-1)}$, with $\{n_4, p_4\} \in \mathbb{N}^*$. In order to relate (15a)–(15d) to Appendix F, note that when $N = r^{4n_4+1}$, $\mathbf{S}^{(4n_4)} = \mathbf{I}_{r^{4n_4}} \otimes \mathbf{P}_r^{(r)} = \mathbf{I}_N$; when $N = r^{4n_4+2}$, $\mathbf{S}^{(4n_4+1)} = \mathbf{I}_{r^{4n_4+1}} \otimes \mathbf{P}_r^{(r)} = \mathbf{I}_N$; and when $N = r^{4n_4+3}$, $\mathbf{S}^{(4n_4+2)} = \mathbf{I}_{r^{4n_4+2}} \otimes \mathbf{P}_r^{(r)} = \mathbf{I}_N$.

B. Mapping to an SDC Pipeline Architecture

Equation (15c) shows that each typical stage of the FFT can be divided in sixteen steps. These steps are similar to those of the radix r^3 . In this case, no new operators appear.

When $r^4 = 2^4$, the implementation of the multiplication by the twiddle factors given by $\mathbf{M3}^{(4i,4)}$ in Step 8 can be simplified. In this case, the twiddle factors given by (37) are the ones in (16), which is shown at the bottom of the page. Therefore, all the operations required by $\mathbf{M3}^{(a,4)}$ can be implemented in a device able to swap the real and imaginary parts of the

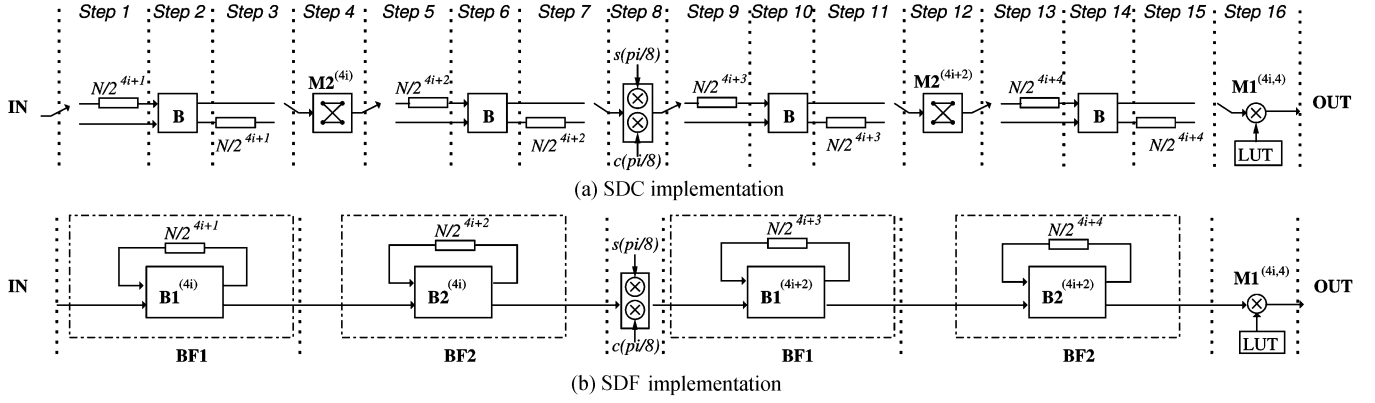
$$\mathbf{T}_N = \begin{cases} \mathbf{Q}_N \cdot \mathbf{T}_N^{(p_4)}, & \text{for } N = r^{4n_4} \\ \mathbf{Q}_N \cdot (\mathbf{S}^{(4n_4)})^{-1} \cdot \mathbf{B} \cdot \mathbf{S}^{(4n_4)} \cdot \mathbf{T}_N^{(p_4)}, & \text{for } N = r^{4n_4+1} \\ \mathbf{Q}_N \cdot (\mathbf{S}^{(4n_4+1)})^{-1} \cdot \mathbf{B} \cdot \mathbf{S}^{(4n_4+1)} \cdot \mathbf{M2}^{(4n_4)} \cdot (\mathbf{S}^{(4n_4)})^{-1} \cdot \mathbf{B} \cdot \mathbf{S}^{(4n_4)} \cdot \mathbf{T}_N^{(p_4)}, & \text{for } N = r^{4n_4+2} \\ \mathbf{Q}_N \cdot (\mathbf{S}^{(4n_4+2)})^{-1} \cdot \mathbf{B} \cdot \mathbf{S}^{(4n_4+2)} \cdot \mathbf{M3}^{(4n_4,3)} \cdot (\mathbf{S}^{(4n_4+1)})^{-1} \cdot \mathbf{B} \cdot \mathbf{S}^{(4n_4+1)} \cdot \mathbf{M2}^{(4n_4)} \cdot (\mathbf{S}^{(4n_4)})^{-1} \cdot \mathbf{B} \cdot \mathbf{S}^{(4n_4)} \cdot \mathbf{T}_N^{(p_4)}, & \text{for } N = r^{4n_4+3} \end{cases} \quad (15a)$$

$$\mathbf{T}_N^{(p_4)} = \prod_{m=0}^{n_4-1} \mathbf{H}^{(n_4-m-1)} \quad (15b)$$

$$\mathbf{H}^{(i)} = \mathbf{M1}^{(4i,4)} \cdot (\mathbf{S}^{(4i+3)})^{-1} \cdot \mathbf{B} \cdot \mathbf{S}^{(4i+3)} \cdot \mathbf{M2}^{(4i+2)} \cdot (\mathbf{S}^{(4i+2)})^{-1} \cdot \mathbf{B} \cdot \mathbf{S}^{(4i+2)} \cdot \mathbf{M3}^{(4i,4)} \cdot (\mathbf{S}^{(4i+1)})^{-1} \cdot \mathbf{B} \cdot \mathbf{S}^{(4i+1)} \cdot \mathbf{M2}^{(4i)} \cdot (\mathbf{S}^{(4i)})^{-1} \cdot \mathbf{B} \cdot \mathbf{S}^{(4i)} \quad (15c)$$

$$\mathbf{Q}_N = \prod_{i=0}^{n_1-1} \mathbf{S}^{(i)} \quad (15d)$$

$$\mathbf{D}_{2^4}^{(2^2)} = \text{diag} \left\{ \begin{pmatrix} 1 & 1 & 1 & 1 & 1 & c(\frac{\pi}{8}) - js(\frac{\pi}{8}) & c(\frac{\pi}{4}) - js(\frac{\pi}{4}) & s(\frac{\pi}{8}) - jc(\frac{\pi}{8}) \\ 1 & c(\frac{\pi}{4}) - js(\frac{\pi}{4}) & -j & (-c(\frac{\pi}{4}) - js(\frac{\pi}{4})) & 1 & s(\frac{\pi}{8}) - jc(\frac{\pi}{8}) & c(\frac{\pi}{4}) - js(\frac{\pi}{4}) & (-c(\frac{\pi}{8}) + js(\frac{\pi}{8})) \end{pmatrix} \right\} \quad (16)$$

Fig. 10. Diagrams of a stage of the radix $r^4 = 2^4$ DIF pipeline architectures.

input data and able to perform two real multiplications by the constant value $\cos(\pi/8)$, two real multiplications by the constant value $\sin(\pi/8)$ and a complex addition [11]. Note that $\cos(\pi/4) = \sin(\pi/4) = 2\sin(\pi/8)\cos(\pi/8)$, and thus, the multiplications by the constant values $\cos(\pi/4)$ and $\sin(\pi/4)$ can be obtained using the former four real multipliers and a shift. The implementation of the i th stage of the DIF SDC architecture when $r^3 = 2^3$ can be seen in Fig. 10(a).

Note that when $N = r^{4n_4}$, no complex multiplier is needed to implement the twiddle factor multiplications given by $\mathbf{M1}^{(4(n_4-1),4)}$ in $\mathbf{H}^{(n_4-1)}$. Once more, the final processing depends on the length of the FFT as defined by (15a).

C. Mapping to an SDF Pipeline Architecture

Steps 1 to 3 and 9 to 11 can be combined into devices $\mathbf{BF1}^{(4i)}$ and $\mathbf{BF1}^{(4i+2)}$, respectively. In addition, Steps 4 to 7 and 12 to 15 can be combined into devices $\mathbf{BF2}^{(4i)}$ and $\mathbf{BF2}^{(4i+2)}$, respectively. Thus, a more efficient SDF architecture is obtained. Fig. 10(b) presents a diagram of the implementation of a stage of this SDF architecture when $r^4 = 2^4$.

VII. MAPPING THE DFT MATRIX TO A RADIX r^k DIF PIPELINE FFT ARCHITECTURE

A. Matrix Representation

The decomposition procedure employed to obtain the former pipeline architectures can be applied to devise new r^k DIF SDC/SDF pipeline architectures with $k > 4$. The matricial representation when $N = r^{(kn_k+l)}$ is given by

$$\mathbf{T}_N = \begin{cases} \mathbf{Q}_N \cdot \mathbf{T}_N^{(p_k)}, & \text{for } l = 0 \\ \mathbf{Q}_N \cdot \mathbf{H}^{(l,k,n_k)} \cdot \mathbf{T}_N^{(p_k)}, & \text{for } l > 0 \end{cases} \quad (17a)$$

$$\mathbf{T}_N^{(p_k)} = \prod_{m=0}^{n_k-1} \mathbf{H}^{(k,k,n_k-m-1)} \quad (17b)$$

$$\mathbf{Q}_N = \prod_{i=0}^{n_1-1} \mathbf{S}^{(i)} \quad (17c)$$

where $p_k = r^{k(n_k-1)}$, with $\{k, n_k, p_k, l\} \in \mathbb{N}^*$ and $l = 0, 1, \dots, k-1$. The term $\mathbf{H}^{(b,k,i)}$ represents the i th

stage of a r^k FFT algorithm. When $b = 1$ and $b = 2$, $\mathbf{H}^{(b,k,i)}$ is obtained replacing i by $k \cdot i/b$ in (10c) and in (11c), respectively. When b is even and $b > 2$, $\mathbf{H}^{(b,k,i)}$ is given by

$$\begin{aligned} \mathbf{H}^{(b,k,i)} = & \mathbf{M1}^{(k \cdot i, k)} \cdot (\mathbf{S}^{(k \cdot i + b - 1)})^{-1} \cdot \mathbf{B} \cdot \mathbf{S}^{(k \cdot i + b - 1)} \\ & \cdot \mathbf{M2}^{(k \cdot i + b - 2)} \cdot (\mathbf{S}^{(k \cdot i + b - 2)})^{-1} \cdot \mathbf{B} \cdot \mathbf{S}^{(k \cdot i + b - 2)} \\ & \cdot \left[\prod_{m=0}^{M=(b/2)-2} [\mathbf{M3}^{(k \cdot i + 2(M-m), b-2(M-m))}] \right. \\ & \cdot (\mathbf{S}^{(k \cdot i + b - 2m - 3)})^{-1} \cdot \mathbf{B} \cdot \mathbf{S}^{(k \cdot i + b - 2m - 3)} \\ & \cdot \mathbf{M2}^{(k \cdot i + b - 2m - 4)} \cdot (\mathbf{S}^{(k \cdot i + b - 2m - 4)})^{-1} \\ & \left. \cdot \mathbf{B} \cdot \mathbf{S}^{(k \cdot i + b - 2m - 4)} \right]. \end{aligned} \quad (17d)$$

When b is odd and $b > 1$, $\mathbf{H}^{(b,k,i)}$ is given by

$$\begin{aligned} \mathbf{H}^{(b,k,i)} = & \mathbf{M1}^{(k \cdot i, k)} \cdot (\mathbf{S}^{(k \cdot i + b - 1)})^{-1} \cdot \mathbf{B} \cdot \mathbf{S}^{(k \cdot i + b - 1)} \\ & \cdot \left[\prod_{m=0}^{M=(b-1)/2-1} [\mathbf{M3}^{(k \cdot i + 2(M-m), b-2(M-m))}] \right. \\ & \cdot (\mathbf{S}^{(k \cdot i + b - 2m - 2)})^{-1} \cdot \mathbf{B} \cdot \mathbf{S}^{(k \cdot i + b - 2m - 2)} \\ & \cdot \mathbf{M2}^{(k \cdot i + b - 2m - 3)} \\ & \left. \cdot (\mathbf{S}^{(k \cdot i + b - 2m - 3)})^{-1} \cdot \mathbf{B} \cdot \mathbf{S}^{(k \cdot i + b - 2m - 3)} \right]. \end{aligned} \quad (17e)$$

This matricial representation is general in terms of N , r and k . When $N = r^{kn_k+l}$ with $l > 0$, (17a) means that, after n_k stages with the structure of a r^k algorithm, an additional processing stage is required with the structure of a r^l algorithm. Thus, we can say that (17a)–(17e) define a kind of mixed-radix algorithm.

B. Mapping to Pipeline Architectures

Equations (17d) and (17e) show that each stage of the FFT $\mathbf{H}^{(b,k,i)}$ can be divided in several steps. In the SDC architectures, each step corresponds to one of the operators defined in Appendix C. No new operators appear. Thus, the r^k algorithms can be mapped to hardware using the kind of elements described in the former sections. Additionally, some steps can be combined in a single device to optimize the use of the commutator and delay line structures, yielding the SDF architectures.

TABLE I
HARDWARE COMPLEXITY OF RADIX r^k PIPELINE ARCHITECTURES WHEN $(N = r^{kn_k+l})$

	Even $k, l=0$	Even k , even l ($l > 0$)	Even k , odd l
M1 C.M.	$n_k - 1$	n_k	n_k
M2 C.M.	$n_k(k/2)$	$n_k(k/2) + (l/2)$	$n_k(k/2) + (l-1)/2$
M3 C.M.	$n_k[(k/2) - 1]$	$n_k[(k/2) - 1] + (l/2) - 1$	$n_k[(k/2) - 1] + (l-1)/2$
M1 T.F.	$r^k(N - r^k)/(r^k - 1)$	$r^k(N - r^l)/(r^k - 1)$	$r^k(N - r^l)/(r^k - 1)$
M2 T.F.	$n_k \cdot r^2(k/2)$	$n_k \cdot r^2(k/2) + r^2(l/2)$	$n_k \cdot r^2(k/2) + r^2(l-1)/2$
M3 T.F.	$n_k \cdot r^2(r^k - r^2)/(r^2 - 1)$	$r^2 [n_k(r^k - r^2)/(r^2 - 1) + (r^l - r^2)/(r^2 - 1)]$	$r^2 [n_k(r^k - r^2)/(r^2 - 1) + (r^l - r)/(r^2 - 1)]$
B	$k \cdot n_k$	$k \cdot n_k + l$	$k \cdot n_k + l$
	Odd $k, l=0$	Odd k , even l ($l > 0$)	Odd k , odd l
M1 C.M.	$n_k - 1$	n_k	n_k
M2 C.M.	$n_k(k-1)/2$	$n_k(k-1)/2 + (l/2)$	$n_k(k-1)/2 + (l-1)/2$
M3 C.M.	$n_k(k-1)/2$	$n_k(k-1)/2 + (l/2) - 1$	$n_k(k-1)/2 + (l-1)/2$
M1 T.F.	$r^k(N - r^k)/(r^k - 1)$	$r^k(N - r^l)/(r^k - 1)$	$r^k(N - r^l)/(r^k - 1)$
M2 T.F.	$n_k \cdot r^2(k-1)/2$	$n_k \cdot r^2(k-1)/2 + r^2(l/2)$	$n_k \cdot r^2(k-1)/2 + r^2(l-1)/2$
M3 T.F.	$n_k \cdot r^2(r^k - r)/(r^2 - 1)$	$r^2 [n_k(r^k - r)/(r^2 - 1) + (r^l - r^2)/(r^2 - 1)]$	$r^2 [n_k(r^k - r)/(r^2 - 1) + (r^l - r)/(r^2 - 1)]$
B	$k \cdot n_k$	$k \cdot n_k + l$	$k \cdot n_k + l$

A stage $\mathbf{H}^{(b,k,i)}$ consists of b butterfly operators $\mathbf{B}$, with their corresponding shuffling and unshuffling $\mathbf{S}$ and $(\mathbf{S})^{-1}$ terms. The hardware that implements the arithmetic of the butterfly operators $\mathbf{B}$ is the same in all the stages of the FFT processor. The length of each of the delay lines used for the shuffling and unshuffling corresponding to the first butterfly of the first stage is N/r . The length of each of these delay lines is reduced by a factor of r from one butterfly to the next one.

A stage of the form $\mathbf{H}^{(k,k,i)}$ has one twiddle factor multiplier operator $\mathbf{M1}$. The number of twiddle factors stored in the LUT used to implement $\mathbf{M1}$ is N for the first stage and it reduces by a factor of r^k from one stage to the next. When $N = r^{kn_k}$, no complex multiplier is needed to implement the twiddle factor multiplications given by $\mathbf{M1}^{(k(n_k-1),k)}$ of the last stage $\mathbf{H}^{(k,k,n_k-1)}$, because $\mathbf{M1}^{(k(n_k-1),k)} = \mathbf{I}_N$. The final processing given by terms of the form $\mathbf{H}^{(l,k,n_k)}$ when $N = r^{kn_k+l}$, with $l \neq 0$, does not actually have a operator $\mathbf{M1}$; i.e., $\mathbf{M1}^{(kn_k,k)} = \mathbf{I}_N$.

For $k > 1$, twiddle factor multiplier operators $\mathbf{M2}$ appear. A stage $\mathbf{H}^{(b,k,i)}$ has $b/2$ operators $\mathbf{M2}$ when b is even and $(b-1)/2$ when b is odd. The number of twiddle factors stored in the LUT used to implement $\mathbf{M2}$ is always r^2 .

For $k > 2$, twiddle factor multiplier operators $\mathbf{M3}$ appear. A stage $\mathbf{H}^{(b,k,i)}$ has $b/2 - 1$ operators $\mathbf{M3}$ when b is even and $(b-1)/2$ when b is odd. The number of twiddle factors stored in the LUT used for the implementation of the first $\mathbf{M3}$ within a stage $\mathbf{H}^{(b,k,i)}$ is r^b and it reduces by a factor of r^2 from one $\mathbf{M3}$ to the next one within the same stage.

VIII. BENEFITS OF THE PROPOSED APPROACH

Radix r^2 , r^3 , and r^4 pipeline FFT architectures are generally presented in the related literature [9]–[12], [14] using summation based expressions or flow graph notations. Neither of these notations is able to give a general expression in N , r , and k that can be easily related to hardware.

The matricial notation has been mainly employed to map the FFT to parallel computer architectures [17]–[22], [25], [27]. They present expressions that are general in terms of N and r . Nevertheless, the algorithms and the mapping they present try to exploit the possible parallelism within a stage, as it adapts better to computer architectures. References [3], [24], and [25]

describe some radix r pipeline architectures using the matricial notation. However, the notation they employ can not be clearly linked to hardware resources and, thus, they do not fully exploit the potential of the matricial notation. Reference [23] studies how the matricial notation of radix r FFT algorithms based upon [15] and [30] maps to different datapath structures.

Sections III–VII have shown that the matricial notation can also be used to represent radix r^k pipeline FFT architectures. The proposed operators enhance the notation so that the resulting representation can be more easily translated to the hardware employed in radix r^k pipeline architectures. The designer only has to write down the matricial representation of the desired algorithm. Then, each operator (or set of operators) corresponds to a hardware element. The parameters to build the hardware element are given by the parameters of the operator.

The proposed approach enables the design of radix r^k SDC/SDF pipeline architectures with new values of r and k . The general expression can be used to evaluate the features of these architectures. From the analysis of the general expression, we can obtain Table I. This table summarizes the overall impact of the arithmetic operators in the hardware complexity of the FFT processor. The implementation of the butterfly depends on the value of r . Therefore, the table only shows the number of butterflies. The number of delay commutator structures used for the shuffling and unshuffling around a butterfly unit is the same as the number of butterfly units. Operators $\mathbf{M1}$, $\mathbf{M2}$, and $\mathbf{M3}$ mainly translate to complex multipliers (C.M.) and LUTs to store the twiddle factors (T.F.). The arithmetic hardware of each butterfly is active $1/r$ of the time during steady-state execution; i.e., when the first input datum of the next FFT is input to the FFT processor just after the last datum of the previous FFT. The complex multipliers used for the implementation of $\mathbf{M1}$, $\mathbf{M2}$, and $\mathbf{M3}$ are active 100% of the time. In order to study the benefits of the new architectures, we analyze the influence of r and k in the reordering operators and the arithmetic operators.

The overall amount of memory words used for the delay lines due to the reordering operators depends neither on r nor k . In a SDC architecture, $2N - 2$ words are used for the delay lines. In a SDF architecture, the amount of memory is $N - 1$ words. Therefore, it can be said that the SDF architecture is more efficient in terms of memory than the SDC architecture and that

TABLE II
HARDWARE COMPLEXITY COMPARISON

Algorithm	Complex Multipliers			Real Constant Multipliers				Twiddle Factors in LUTs			Complex Adders				
	M1	M3	Total	M1	M2	M3	Total	M1	M3	Total	M1	M2	M3	B	Total
R2SDC [6]/SDF [7]	8	-	8	14	-	-	14	32701	-	32701	3	-	-	28	31
R2 ² SDF [8]	5	-	5	4	-	-	4	21824	-	21824	1	-	-	28	29
R2 ³ SDF [9]	3	-	3	8	-	8	8	18688	-	18688	-	-	4	28	32
R2 ⁴ SDF [10]	2	-	2	-	-	12	12	17468	-	17468	-	-	3	28	31
R2 ⁵ SDF	2	-	2	-	-	24	24	16896	-	16896	-	-	5	28	33
R2 ⁶ SDF	2	4	6	-	-	8	8	16640	128	16768	-	-	2	28	30
R2 ⁷ SDF	1	2	3	-	-	20	20	16384	256	16640	-	-	4	28	32
R2 ⁸ SDF	1	3	4	-	-	8	8	16384	384	16768	-	-	2	28	30
R2 ⁹ SDF	1	2	3	-	-	20	20	16384	640	17024	-	-	4	28	32
R2 ¹⁰ SDF	1	3	4	-	-	8	8	16384	1344	17728	-	-	2	28	30
R2 ¹¹ SDF	1	3	4	-	-	12	12	16384	2688	19072	-	-	3	28	31
R2 ¹² SDF	1	4	5	-	-	4	4	16384	5440	21824	-	-	1	28	29
R2 ¹³ SDF	1	4	5	-	-	10	10	16384	10880	27264	-	-	2	28	30
R2 ¹⁴ SDF	-	5	5	-	-	4	4	-	21824	21824	-	-	1	28	29
R4SDC [6]/SDF [7]	5	-	5	4	-	-	4	21824	-	21824	1	-	-	56	57
R4 ² SDC [11]	2	-	2	-	12	-	12	17376	96	17472	-	3	-	56	59
R4 ² SDF	2	-	2	-	12	-	12	17376	96	17472	-	3	-	56	59
R4 ³ SDF	2	2	4	-	8	-	8	16640	128	16768	-	2	-	56	58
R4 ⁴ SDF	1	2	3	-	12	-	12	16384	320	16704	-	3	-	56	59
R4 ⁵ SDF	1	2	3	-	12	-	12	16384	1088	17472	-	3	-	56	59
R4 ⁶ SDF	1	2	3	-	12	-	12	16384	4352	20736	-	3	-	56	59
R4 ⁷ SDF	-	2	2	-	12	-	12	-	17472	17472	-	3	-	56	59

the overall amount of memory words is fixed by the number of points of the FFT. $N - 1$ is the minimum amount of memory reported in the literature for a pipeline FFT architecture. Modifying the values of r or k will not introduce any penalty in the amount of memory words used for the reordering operators.

From the results shown in Table I, we can observe that increasing the value of r for fixed values of N and k will reduce the number of butterflies and the overall number of complex multipliers. However, the butterflies become more complex. Regarding the number of twiddle factors, we can observe that the number of twiddle factors due to **M1** are reduced, while the number of twiddle factors due to **M2** and **M3** increase. For small values of k , we have a reduction of the overall number of twiddle factors, while for large values of k we have an increase in the overall number of twiddle factors. Thus, the benefit of increasing the value of r depends on the value of k .

If we fix the values of N and r , we can study the benefits of increasing k . The overall number of butterflies is $\log_r N$, and thus, increasing k does not introduce any penalty in the hardware due to the butterflies. Increasing the value of k reduces the number of complex multipliers due to **M1**. Once **M2** and **M3** appear in an architecture, the number of complex multipliers due to each one remains approximately constant. Thus, an important benefit of increasing the value of k is that it reduces the overall number of complex multipliers. When k increases, the number of twiddle factors due to **M1** is reduced. For $\lceil (\log_r N)/2 \rceil \leq k < \log_r N$, we have one single **M1** with N twiddle factors. Finally, when $k = \log_r N$, **M1** = **I_N** and no hardware is needed to implement any **M1**. The number of twiddle factors due to **M2** remains constant with k . The number of twiddle factors due to **M3** increases with k . Adding the contributions of **M1**, **M2** and **M3**, we can observe that the overall number of twiddle factors first reduces with k and then it starts to increase. We can conclude that an optimum value of k exists for each value of N and r . Usually this optimum value is near $k = \lceil (\log_r N)/2 \rceil$.

From the above, we can conclude that architectures with larger values of r and k may yield more area efficient circuits than those in the literature. The results in Table I enable the designer to look for those values of r and k that result in an optimum single-path FFT pipeline architecture for a given application. This search can be easily performed thanks to the close link between the proposed matricial representation and implementation. No other notation allows such an exploration within the pipeline architectures. In Section VIII-A, an example of single-path pipeline FFT architecture selection is presented and the proposed novel architectures are compared to the literature.

A. Case Study: $N = 16K$

We concentrate the analysis on $r = 2^k$ and $r = 4^k$ algorithms, which result in simple multiplier-less butterflies. For the hardware complexity analysis we state the following model of implementation. A $r = 2^k$ butterfly can be implemented using two complex adders and a $r = 4^k$ butterfly can be implemented using eight complex adders. For the radix $r = 2^k$ algorithms, **M2** is implemented as described in Section IV. When the number of twiddle factors corresponding to a twiddle factor multiplier operator is 8 or 16, its implementation can be optimized as described in Sections V and VI, respectively. When the number of twiddle factors corresponding to a twiddle factor multiplier operator is 32, it can be implemented using eight real multiplications by a constant value and a complex addition.

Table II compares the overall hardware complexity of the proposed radix 2^k and 4^k pipeline FFT architectures when $N = 2^{14} = 16384$ with other single-path pipeline architectures in the literature. When necessary, the works in the literature have been modified to accommodate them to $N = 16K$ using the general expression proposed in Section VII.

The table shows the number of complex multipliers, the number of real constant multipliers and the number of twiddle

factors stored in LUTs for each type of twiddle factor multiplier operator. The number of complex adders used in each arithmetic operator is also shown. Among the architectures proposed in this paper, SDF architectures have been selected as they halve the delay line memory requirements.

The results in the table show that the minimum number of complex multipliers is achieved by R2⁴SDF, R2⁵SDF, R4²SDC/SDF, and R4⁷SDF. The minimum number of LUT twiddle factors is obtained by R2⁷SDF. Among radix $r = 4$ algorithms, R4⁴SDF has the minimum number of LUT twiddle factors.

The R2⁵SDF saves 572 LUT twiddle factors with respect to R2⁴SDF at the expense of 12 additional constant multipliers and two additional complex adders. The R2⁷SDF saves 828 LUT twiddle factors with respect to R2⁴SDF at the expense of one additional complex multiplier, eight additional constant multipliers and one additional complex adder. If the saving in LUT twiddle factors compensates the additional arithmetic complexity, R2⁷SDF becomes the most appropriate pipeline architecture when $r = 2$.

The R4²SDF halves the delay line memory requirements of R4²SDC. The R4⁴SDF saves 673 LUT twiddle factors with respect to R4²SDF and R4⁷SDF, at the expense of one complex multiplier. Thus, R4⁴SDF is the most appropriate pipeline architecture when $r = 4$.

Note that for both $r = 2$ and $r = 4$, the most area-efficient r^k single-path pipeline architecture is attained when $k = \lceil (\log_r N)/2 \rceil$. If we compare R2⁷SDF with R4⁴SDF, it can be seen that R2⁷SDF saves 64 LUT twiddle factors and 27 complex adders, at the expense of only eight additional real constant multipliers. Both architectures improve the single-path pipeline architectures in the literature. Between R2⁷SDF and R4⁴SDF, R2⁷SDF is the best architecture for $N = 16K$.

IX. CONCLUSION

The matricial approach can be applied to analyze the radix r^k SDC/SDF pipeline architectures. The corresponding FFT algorithms can be described using the reduced set of operators proposed in this paper. This set of operators can be mapped to equivalent hardware circuits, so that the designer can easily go from the matricial representations to their implementations and vice versa. Possible mappings of these operators to hardware circuits are shown for radix r , r^2 , r^3 , and r^4 ; and the details of the corresponding SDC/SDF pipeline FFT architectures are presented. Both SDC and SDF pipeline architectures are represented by the same matricial expression: it is the mapping of the operators to different hardware circuits what makes the resulting architecture either SDC or SDF.

The matricial expression provides a compact and general representation of the SDC/SDF radix r^k pipeline architectures. These expressions are general in terms of r and N . The derivation of the matricial representations leads to general expressions, where N is a power of r , but not necessarily equal to r^{kn} . Furthermore, a general expression for the SDC/SDF radix r^k pipeline architectures is given. The case study presented shows that the increase of the exploration range in r and k enables the designer to achieve new pipeline architectures, more area efficient than those available in the literature.

APPENDIX A SUMMARY OF PROPERTIES

This appendix summarizes some useful properties of the Kronecker product and the stride permutations matrices.

Lemma 1: Let $\mathbf{A}$, $\mathbf{B}$, $\mathbf{C}$, $\mathbf{D}$ be square matrices, then

$$(\mathbf{A} \otimes \mathbf{B}) \cdot (\mathbf{C} \otimes \mathbf{D}) = (\mathbf{A} \cdot \mathbf{C}) \otimes (\mathbf{B} \cdot \mathbf{D}). \quad (18)$$

Lemma 2: Let M be divisible by m and $\mathbf{A}$, $\mathbf{B}$ square matrices of size M/m , then

$$(\mathbf{A}_{M/m} \cdot \mathbf{B}_{M/m}) \otimes \mathbf{I}_m = (\mathbf{A}_{M/m} \otimes \mathbf{I}_m) \cdot (\mathbf{B}_{M/m} \otimes \mathbf{I}_m) \quad (19a)$$

$$\mathbf{I}_m \otimes (\mathbf{A}_{M/m} \cdot \mathbf{B}_{M/m}) = (\mathbf{I}_m \otimes \mathbf{A}_{M/m}) \cdot (\mathbf{I}_m \otimes \mathbf{B}_{M/m}). \quad (19b)$$

Lemma 3: Let M be divisible by m and $\mathbf{A}$ be a square matrix of size M/m , then

$$[\mathbf{A}_{M/m} \otimes \mathbf{I}_m] = \mathbf{P}_M^{(m)} \cdot [\mathbf{I}_m \otimes \mathbf{A}_{M/m}] \cdot (\mathbf{P}_M^{(m)})^{-1}. \quad (20)$$

Lemma 4: Let M be divisible by m , then

$$(\mathbf{P}_M^{(m)})^{-1} = \mathbf{P}_M^{(M/m)} = (\mathbf{P}_M^{(m)})^T. \quad (21)$$

Lemma 5: Let M be divisible by $m_1 \cdot m_2 \cdot m_3 \cdots m_k$, then

$$\mathbf{P}_M^{(m_1)} \cdot \mathbf{P}_M^{(m_2)} \cdot \mathbf{P}_M^{(m_3)} \cdots \mathbf{P}_M^{(m_k)} = \mathbf{P}_M^{(m_1 \cdot m_2 \cdot m_3 \cdots m_k)}. \quad (22)$$

Lemma 6: Let $\mathbf{A}$ be a diagonal matrix and $\mathbf{B}$ a permutation symmetric matrix, then

$$\mathbf{A} \cdot \mathbf{B} = \mathbf{B} \cdot \mathbf{A}. \quad (23)$$

Lemma 7: Let M be divisible by m^{ki+k} , then

$$\prod_{z=0}^{k-1} [\mathbf{I}_{m^{ki+z}} \otimes \mathbf{P}_{M/m^{ki+z}}^{(m)}] = [\mathbf{I}_{m^{ki}} \otimes \mathbf{P}_{M/m^{ki}}^{(m^k)}] \cdot \prod_{z=0}^{k-2} [\mathbf{I}_{m^{(k-z)i}} \otimes \mathbf{P}_{m^{k-z}}^{(m^{k-z-1})} \otimes \mathbf{I}_{M/m^{(k-z)(i+1)}}]. \quad (24)$$

APPENDIX B

RADIX r DIF SDC/SDF FFT ALGORITHM DERIVATION

Applying Lemmas 3 and 4, (8) can be written as

$$\mathbf{T}_N = \mathbf{P}_N^{(r)} \cdot [\mathbf{I}_r \otimes \mathbf{T}_{N/r}] \cdot \mathbf{D}_N^{(r)} \cdot (\mathbf{P}_N^{(r)})^{-1} \cdot [\mathbf{I}_{N/r} \otimes \mathbf{T}_r] \cdot \mathbf{P}_{N/r}^{(r)}. \quad (25)$$

This is the basic recursive decomposition of the radix r DIF SDC/SDF pipeline FFT algorithms. This decomposition can be applied again to the term $\mathbf{T}_{(N/r)}$ in (25):

$$\mathbf{T}_{N/r} = \mathbf{P}_{N/r}^{(r)} \cdot [\mathbf{I}_r \otimes \mathbf{T}_{N/r^2}] \cdot \mathbf{D}_{N/r}^{(r)} \cdot (\mathbf{P}_{N/r}^{(r)})^{-1} \cdot [\mathbf{I}_{N/r^2} \otimes \mathbf{T}_r] \cdot \mathbf{P}_{N/r^2}^{(r)}. \quad (26)$$

Replacing (26) into (25), we have

$$\begin{aligned} \mathbf{T}_N &= \mathbf{P}_N^{(r)} \cdot [\mathbf{I}_r \otimes \mathbf{P}_{N/r}^{(r)}] \cdot [\mathbf{I}_{r^2} \otimes \mathbf{T}_{N/r^2}] \\ &\quad \cdot [\mathbf{I}_r \otimes \mathbf{D}_{N/r}^{(r)}] \cdot [\mathbf{I}_r \otimes (\mathbf{P}_{N/r}^{(r)})^{-1}] \\ &\quad \cdot [\mathbf{I}_{N/r} \otimes \mathbf{T}_r] \cdot [\mathbf{I}_r \otimes \mathbf{P}_{N/r}^{(r)}] \\ &\quad \cdot \mathbf{D}_N^{(r)} \cdot (\mathbf{P}_N^{(r)})^{-1} \cdot [\mathbf{I}_{N/r} \otimes \mathbf{T}_r] \cdot \mathbf{P}_N^{(r)}. \end{aligned} \quad (27)$$

If we iterate $k - 1$ times, we obtain

$$\mathbf{T}_N = \mathbf{C}^{(0)} \cdot \mathbf{C}^{(1)} \cdot \mathbf{C}^{(2)} \dots \mathbf{C}^{(k-1)} \cdot [\mathbf{I}_{r^k} \otimes \mathbf{T}_{N/r^k}] \cdot \mathbf{H}^{(k-1)} \cdot \mathbf{H}^{(k-2)} \cdot \mathbf{H}^{(k-3)} \dots \mathbf{H}^{(0)} \quad (28)$$

where

$$\mathbf{C}^{(i)} = [\mathbf{I}_{r^i} \otimes \mathbf{P}_{N/r^i}^{(r)}] \quad (29)$$

$$\begin{aligned} \mathbf{H}^{(i)} &= [\mathbf{I}_{r^i} \otimes \mathbf{D}_{N/r^i}^{(r)}] \cdot [\mathbf{I}_{r^i} \otimes (\mathbf{P}_{N/r^i}^{(r)})^{-1}] \\ &\quad \cdot [\mathbf{I}_{N/r} \otimes \mathbf{T}_r] \cdot [\mathbf{I}_{r^i} \otimes \mathbf{P}_{N/r^i}^{(r)}]. \end{aligned} \quad (30)$$

When $N = r^k$, then $T_{N/r^k} = T_1 = 1$ and the iterations are over. Thus, we can write

$$\mathbf{T}_N = \mathbf{Q}_N \cdot \mathbf{T}_N^{(p_1)} \quad (31a)$$

$$\mathbf{T}_N^{(p_1)} = \prod_{m=0}^{n_1-1} \mathbf{H}^{(n_1-m-1)} \quad (31b)$$

$$\mathbf{Q}_N = \prod_{i=0}^{n_1-1} \mathbf{C}^{(i)} \quad (31c)$$

with $N = r^{n_1}$, $p_1 = r^{n_1-1}$ and $\{n_1, p_1\} \in \mathbb{N}^*$.

APPENDIX C SUMMARY OF OPERATORS

In order to simplify the matrix notation, nine operators are defined. Two different types of operators can be distinguished: the reordering operators and the arithmetic operators.

A. Reordering Operators

- *Shuffling Operator* $\mathbf{S}^{(a)}$: Let N be divisible by r^a ,

$$\mathbf{S}^{(a)} = \mathbf{I}_{r^a} \otimes \mathbf{P}_{N/r^a}^{(r)}. \quad (32)$$

Note that

$$(\mathbf{S}^{(a)})^{-1} = \mathbf{I}_{r^a} \otimes (\mathbf{P}_{N/r^a}^{(r)})^{-1}. \quad (33)$$

B. Arithmetic Operators

- *Butterfly Operator* $\mathbf{B}$: Let N be divisible by r ,

$$\mathbf{B} = \mathbf{I}_{N/r} \otimes \mathbf{T}_r. \quad (34)$$

- *First Twiddle Factor Multiplier Operator* $\mathbf{M1}^{(a,b)}$: Let N be divisible by r^{a+b} , then $\mathbf{M1}^{(a,b)}$ is defined by (35), shown at the bottom of the page.
- *Second Twiddle Factor Multiplier Operator* $\mathbf{M2}^{(a)}$: Let N be divisible by r^{a+2} ,

$$\mathbf{M2}^{(a)} = [\mathbf{I}_{r^a} \otimes \mathbf{D}_{r^2}^{(r)} \otimes \mathbf{I}_{N/r^{a+2}}]. \quad (36)$$

- *Third Twiddle Factor Multiplier Operator* $\mathbf{M3}^{(a,b)}$: Let N be divisible by r^{a+b} and $b \geq 2$,

$$\begin{aligned} \mathbf{M3}^{(a,b)} &= \mathbf{I}_{r^a} \otimes ([\mathbf{P}_{r^2}^{(r)} \otimes \mathbf{I}_{r^{(b-2)}}] \\ &\quad \cdot \mathbf{D}_{r^b}^{(r^2)} \cdot [\mathbf{P}_{r^2}^{(r)} \otimes \mathbf{I}_{r^{(b-2)}}]) \otimes \mathbf{I}_{N/r^{a+b}}. \end{aligned} \quad (37)$$

APPENDIX D

RADIX r^2 DIF SDC/SDF FFT ALGORITHM DERIVATION

We rewrite (8) as if we were to decompose $\mathbf{T}_N$ using butterflies of a r^2 algorithm:

$$\mathbf{T}_N = \mathbf{P}_N^{(r^2)} \cdot [\mathbf{I}_{r^2} \otimes \mathbf{T}_{N/r^2}] \cdot \mathbf{D}_N^{(r^2)} \cdot [\mathbf{T}_{r^2} \otimes \mathbf{I}_{N/r^2}]. \quad (38)$$

In order to avoid the complexity of a radix r^2 butterfly, we further decompose the term $\mathbf{T}_{r^2}$ into simpler butterflies using (8):

$$\mathbf{T}_{r^2} = \mathbf{P}_{r^2}^{(r)} \cdot [\mathbf{I}_r \otimes \mathbf{T}_r] \cdot \mathbf{D}_{r^2}^{(r)} \cdot [\mathbf{T}_r \otimes \mathbf{I}_r]. \quad (39)$$

Replacing (39) into (38), we have

$$\begin{aligned} \mathbf{T}_N &= \mathbf{P}_N^{(r^2)} \cdot [\mathbf{I}_{r^2} \otimes \mathbf{T}_{N/r^2}] \cdot \mathbf{D}_N^{(r^2)} \cdot [\mathbf{P}_{r^2}^{(r)} \otimes \mathbf{I}_{N/r^2}] \\ &\quad \cdot [\mathbf{I}_r \otimes \mathbf{T}_r \otimes \mathbf{I}_{N/r^2}] \cdot [\mathbf{D}_{r^2}^{(r)} \otimes \mathbf{I}_{N/r^2}] \\ &\quad \cdot [\mathbf{T}_r \otimes \mathbf{I}_r \otimes \mathbf{I}_{N/r^2}]. \end{aligned} \quad (40)$$

Applying Lemmas 3, 4 and 6 to (40), we arrive to

$$\begin{aligned} \mathbf{T}_N &= \mathbf{P}_N^{(r^2)} \cdot [\mathbf{P}_{r^2}^{(r)} \otimes \mathbf{I}_{N/r^2}] \cdot [\mathbf{I}_{r^2} \otimes \mathbf{T}_{N/r^2}] \\ &\quad \cdot [\mathbf{P}_{r^2}^{(r)} \otimes \mathbf{I}_{N/r^2}] \cdot \mathbf{D}_N^{(r^2)} \cdot [\mathbf{P}_{r^2}^{(r)} \otimes \mathbf{I}_{N/r^2}] \\ &\quad \cdot [\mathbf{I}_r \otimes (\mathbf{P}_{N/r}^{(r)})^{-1}] \cdot [\mathbf{I}_{N/r} \otimes \mathbf{T}_r] \cdot [\mathbf{I}_r \otimes \mathbf{P}_{N/r}^{(r)}] \\ &\quad \cdot [\mathbf{D}_{r^2}^{(r)} \otimes \mathbf{I}_{N/r^2}] \cdot (\mathbf{P}_N^{(r)})^{-1} \cdot [\mathbf{I}_{N/r} \otimes \mathbf{T}_r] \cdot \mathbf{P}_N^{(r)}. \end{aligned} \quad (41)$$

$$\mathbf{M1}^{(a,b)} = \begin{cases} \mathbf{I}_{r^a} \otimes \left(\left(\prod_{l=1}^b [\mathbf{P}_{r^l}^{(r)} \otimes \mathbf{I}_{N/r^{a+l}}] \right) \cdot \mathbf{D}_{N/r^a}^{(r^b)} \cdot \left(\prod_{l=1}^b [\mathbf{P}_{r^l}^{(r)} \otimes \mathbf{I}_{N/r^{a+l}}] \right) \right), & \text{if } r^b < \frac{N}{r^a} \\ \mathbf{I}_N, & \text{if } r^b \geq \frac{N}{r^a} \end{cases} \quad (35)$$

This is the basic recursive decomposition of the radix r^2 DIF SDC/SDF pipeline FFT algorithms.

If we apply the decomposition in (41) $k-1$ times, we obtain

$$\mathbf{T}_N = \mathbf{C}^{(0)} \cdot \mathbf{C}^{(1)} \cdot \mathbf{C}^{(2)} \dots \mathbf{C}^{(k-1)} \cdot [\mathbf{I}_{r^{2k}} \otimes \mathbf{T}_{N/r^{2k}}] \cdot \mathbf{H}^{(k-1)} \cdot \mathbf{H}^{(k-2)} \cdot \mathbf{H}^{(k-3)} \dots \mathbf{H}^{(0)} \quad (42)$$

where

$$\mathbf{C}^{(i)} = [\mathbf{I}_{r^{2i}} \otimes \mathbf{P}_{N/r^{2i}}^{(r^2)} \cdot [\mathbf{I}_{r^{2i}} \otimes \mathbf{P}_{r^2}^{(r)} \otimes \mathbf{I}_{N/r^{(2i+2)}}]] \quad (43)$$

$$\begin{aligned} \mathbf{H}^{(i)} = & \mathbf{I}_{r^{2i}} \otimes ([\mathbf{P}_{r^2}^{(r)} \otimes \mathbf{I}_{N/r^{2i+2}}] \cdot \mathbf{D}_{N/r^{2i}} \\ & \cdot [\mathbf{P}_{r^2}^{(r)} \otimes \mathbf{I}_{N/r^{2i+2}}]) \\ & \cdot [\mathbf{I}_{r^{2i+1}} \otimes (\mathbf{P}_{N/r^{2i+1}}^{(r)})^{-1}] \cdot [\mathbf{I}_{N/r} \otimes \mathbf{T}_r] \\ & \cdot [\mathbf{I}_{r^{2i+1}} \otimes \mathbf{P}_{N/r^{2i+1}}^{(r)}] \cdot [\mathbf{I}_{r^{2i}} \otimes \mathbf{D}_{r^2}^{(r)} \otimes \mathbf{I}_{N/r^{2i+2}}] \\ & \cdot [\mathbf{I}_{r^{2i}} \otimes (\mathbf{P}_{N/r^{2i}}^{(r)})^{-1}] \cdot [\mathbf{I}_{N/r} \otimes \mathbf{T}_r] \\ & \cdot [\mathbf{I}_{r^{2i}} \otimes \mathbf{P}_{N/r^{2i}}^{(r)}]. \end{aligned} \quad (44)$$

When $N = r^{2k}$, then $\mathbf{T}_{N/r^{2k}} = \mathbf{T}_1 = 1$ and the iterations are over. When $N = r^{2k+1}$, then $\mathbf{T}_{N/r^{2k}} = \mathbf{T}_r$ and the iterations are over as well, because no further decomposition using radix r is possible. Thus,

$$\mathbf{T}_N = \begin{cases} \mathbf{Q}_N \cdot \mathbf{T}_N^{(p_2)}, & \text{for } N = r^{2n_2} \\ \mathbf{Q}_N \cdot [\mathbf{I}_{N/r} \otimes \mathbf{T}_r] \cdot \mathbf{T}_N^{(p_2)}, & \text{for } N = r^{2n_2+1} \end{cases} \quad (45a)$$

$$\mathbf{T}_N^{(p_2)} = \prod_{m=0}^{n_2-1} \mathbf{H}^{(n_2-m-1)} \quad (45b)$$

$$\mathbf{Q}_N = \prod_{i=0}^{n_2-1} \mathbf{C}^{(i)}, \quad (45c)$$

where $p_2 = r^{2(n_2-1)}$ and $\{n_2, p_2\} \in \mathbb{N}^*$. Using Lemma 7, (43) can be further manipulated so that (45c) is written as

$$\mathbf{Q}_N = \prod_{i=0}^{n_2-1} [\mathbf{I}_{r^i} \otimes \mathbf{P}_{N/r^i}^{(r)}]. \quad (46)$$

APPENDIX E

RADIX DIF r^3 FFT ALGORITHM DERIVATION

We rewrite (8) as if we were to decompose $\mathbf{T}_N$ using butterflies of a r^3 algorithm:

$$\mathbf{T}_N = \mathbf{P}_N^{(r^3)} \cdot [\mathbf{I}_{r^3} \otimes \mathbf{T}_{N/r^3}] \cdot \mathbf{D}_N^{(r^3)} \cdot [\mathbf{T}_{r^3} \otimes \mathbf{I}_{N/r^3}]. \quad (47)$$

We further decompose the term $\mathbf{T}_{r^3}$ into simpler butterflies using (8) and (39):

$$\begin{aligned} \mathbf{T}_{r^3} = & \mathbf{P}_{r^3}^{(r^2)} \cdot [\mathbf{I}_{r^2} \otimes \mathbf{T}_r] \cdot \mathbf{D}_{r^3}^{(r^2)} \cdot [\mathbf{P}_{r^2}^{(r)} \otimes \mathbf{I}_r] \\ & \cdot [\mathbf{I}_r \otimes \mathbf{T}_r \otimes \mathbf{I}_r] \cdot [\mathbf{D}_{r^2}^{(r)} \otimes \mathbf{I}_r] \cdot [\mathbf{T}_r \otimes \mathbf{I}_{r^2}]. \end{aligned} \quad (48)$$

Applying Lemmas 3, 6 and 7, (48) can be rearranged as

$$\begin{aligned} \mathbf{T}_{r^3} = & [\mathbf{P}_{r^2}^{(r)} \otimes \mathbf{I}_r] \cdot \mathbf{P}_{r^3}^{(r)} \cdot [\mathbf{I}_{r^2} \otimes \mathbf{T}_r] \cdot [\mathbf{P}_{r^2}^{(r)} \otimes \mathbf{I}_r] \cdot \mathbf{D}_{r^3}^{(r^2)} \\ & \cdot [\mathbf{P}_{r^2}^{(r)} \otimes \mathbf{I}_r] \cdot [\mathbf{I}_r \otimes \mathbf{T}_r \otimes \mathbf{I}_r] \cdot [\mathbf{D}_{r^2}^{(r)} \otimes \mathbf{I}_r] \cdot [\mathbf{T}_r \otimes \mathbf{I}_{r^2}]. \end{aligned} \quad (49)$$

Replacing (49) in (47)

$$\begin{aligned} \mathbf{T}_N = & \mathbf{P}_N^{(r^3)} \cdot [\mathbf{I}_{r^3} \otimes \mathbf{T}_{N/r^3}] \cdot \mathbf{D}_N^{(r^3)} \cdot [\mathbf{P}_{r^2}^{(r)} \otimes \mathbf{I}_{N/r^2}] \\ & \cdot [\mathbf{P}_{r^3}^{(r)} \otimes \mathbf{I}_{N/r^3}] \cdot [\mathbf{I}_{r^2} \otimes \mathbf{T}_r \otimes \mathbf{I}_{N/r^3}] \\ & \cdot [\mathbf{P}_{r^2}^{(r)} \otimes \mathbf{I}_{N/r^2}] \cdot [\mathbf{D}_{r^3}^{(r^2)} \otimes \mathbf{I}_{N/r^3}] \\ & \cdot [\mathbf{P}_{r^2}^{(r)} \otimes \mathbf{I}_{N/r^2}] \cdot [\mathbf{I}_r \otimes \mathbf{T}_r \otimes \mathbf{I}_{N/r^2}] \\ & \cdot [\mathbf{D}_{r^2}^{(r)} \otimes \mathbf{I}_{N/r^2}] \cdot [\mathbf{T}_r \otimes \mathbf{I}_{N/r}]. \end{aligned} \quad (50)$$

Applying Lemmas 3, 4, and 6 to (50) and after some lengthy manipulations, we arrive to the basic recursive decomposition of the radix r^3 DIF SDC/SDF pipeline FFT algorithms:

$$\begin{aligned} \mathbf{T}_N = & \mathbf{P}_N^{(r^3)} \cdot [\mathbf{P}_{r^3}^{(r^2)} \otimes \mathbf{I}_{N/r^3}] \cdot [(\mathbf{P}_{r^2}^{(r)})^{-1} \otimes \mathbf{I}_{N/r^2}] \\ & \cdot [\mathbf{I}_{r^3} \otimes \mathbf{T}_{N/r^3}] \cdot \mathbf{E}_N^{(r)} \cdot [\mathbf{I}_{r^2} \otimes (\mathbf{P}_{N/r^2}^{(r)})^{-1}] \\ & \cdot [\mathbf{I}_{N/r} \otimes \mathbf{T}_r] \cdot [\mathbf{I}_{r^2} \otimes \mathbf{P}_{N/r^2}^{(r)}] \cdot [\mathbf{P}_{r^2}^{(r)} \otimes \mathbf{I}_{N/r^2}] \\ & \cdot [\mathbf{D}_{r^3}^{(r^2)} \otimes \mathbf{I}_{N/r^3}] \cdot [\mathbf{P}_{r^2}^{(r)} \otimes \mathbf{I}_{N/r^2}] \\ & \cdot [\mathbf{I}_r \otimes (\mathbf{P}_{N/r}^{(r)})^{-1}] \cdot [\mathbf{I}_{N/r} \otimes \mathbf{T}_r] \cdot [\mathbf{I}_r \otimes \mathbf{P}_{N/r}^{(r)}] \\ & \cdot [\mathbf{D}_{r^2}^{(r)} \otimes \mathbf{I}_{N/r^2}] \cdot (\mathbf{P}_N^{(r)})^{-1} \cdot [\mathbf{I}_{N/r} \otimes \mathbf{T}_r] \cdot \mathbf{P}_N^{(r)} \end{aligned} \quad (51)$$

where

$$\begin{aligned} \mathbf{E}_N^{(r)} = & [\mathbf{P}_{r^2}^{(r)} \otimes \mathbf{I}_{N/r^2}] \cdot [\mathbf{P}_{r^3}^{(r)} \otimes \mathbf{I}_{N/r^3}] \\ & \cdot \mathbf{D}_N^{(r^3)} \cdot [\mathbf{P}_{r^2}^{(r)} \otimes \mathbf{I}_{N/r^2}] \cdot [\mathbf{P}_{r^3}^{(r)} \otimes \mathbf{I}_{N/r^3}]. \end{aligned} \quad (52)$$

If we apply the decomposition in (51) $k-1$ times, we obtain

$$\mathbf{T}_N = \mathbf{C}^{(0)} \cdot \mathbf{C}^{(1)} \cdot \mathbf{C}^{(2)} \dots \mathbf{C}^{(k-1)} \cdot [\mathbf{I}_{r^{3k}} \otimes \mathbf{T}_{N/r^{3k}}] \cdot \mathbf{H}^{(k-1)} \cdot \mathbf{H}^{(k-2)} \cdot \mathbf{H}^{(k-3)} \dots \mathbf{H}^{(0)} \quad (53)$$

with

$$\begin{aligned} \mathbf{C}^{(i)} = & [\mathbf{I}_{r^{3i}} \otimes \mathbf{P}_{N/r^{3i}}^{(r^3)}] \cdot [\mathbf{I}_{r^{3i}} \otimes \mathbf{P}_{r^3}^{(r^2)} \otimes \mathbf{I}_{N/r^{(3i+3)}}] \\ & \cdot [\mathbf{I}_{r^{2i}} \otimes \mathbf{P}_{r^2}^{(r)} \otimes \mathbf{I}_{N/r^{(2i+2)}}] \end{aligned} \quad (54)$$

$$\begin{aligned} \mathbf{H}^{(i)} = & [\mathbf{I}_{r^{3i}} \otimes \mathbf{E}_{N/r^{3i}}^{(r)}] \cdot [\mathbf{I}_{r^{3i+2}} \otimes (\mathbf{P}_{N/r^{3i+2}}^{(r)})^{-1}] \\ & \cdot [\mathbf{I}_{N/r} \otimes \mathbf{T}_r] \cdot [\mathbf{I}_{r^{3i+2}} \otimes \mathbf{P}_{N/r^{3i+2}}^{(r)}] \\ & \cdot [\mathbf{I}_{r^{3i}} \otimes [(\mathbf{P}_{r^2}^{(r)} \otimes \mathbf{I}_r) \cdot \mathbf{D}_{r^3}^{(r^2)} \cdot (\mathbf{P}_{r^2}^{(r)} \otimes \mathbf{I}_r)] \\ & \quad \otimes \mathbf{I}_{N/r^{3i+3}}] \cdot [\mathbf{I}_{r^{3i+1}} \otimes (\mathbf{P}_{N/r^{3i+1}}^{(r)})^{-1}] \\ & \cdot [\mathbf{I}_{N/r} \otimes \mathbf{T}_r] \cdot [\mathbf{I}_{r^{3i+1}} \otimes \mathbf{P}_{N/r^{3i+1}}^{(r)}] \\ & \cdot [\mathbf{I}_{r^{3i}} \otimes \mathbf{D}_{r^2}^{(r)} \otimes \mathbf{I}_{N/r^{3i+2}}] \cdot [\mathbf{I}_{r^{3i}} \otimes (\mathbf{P}_{N/r^{3i}}^{(r)})^{-1}] \\ & \cdot [\mathbf{I}_{N/r} \otimes \mathbf{T}_r] \cdot [\mathbf{I}_{r^{3i}} \otimes \mathbf{P}_{N/r^{3i}}^{(r)}]. \end{aligned} \quad (55)$$

When $N = r^{3k}$, then $\mathbf{T}_{N/r^{3k}} = \mathbf{T}_1 = 1$ and the iterations are over. When $N = r^{3k+1}$, then $\mathbf{T}_{N/r^{3k}} = \mathbf{T}_r$ and the iterations are over as well, because no further decomposition using radix r is possible. When $N = r^{3k+2}$, then $\mathbf{T}_{N/r^{3k}} = \mathbf{T}_{r^2}$ and it must be decomposed in terms of $\mathbf{T}_r$. Thus, we can write equations (56a)–(56c) shown at the bottom of the next page, where $p_3 = r^{3(n_3-1)}$ and $\{n_3, p_3\} \in \mathbb{N}^*$. Using Lemma 7, (54) can be further manipulated so that (56c) is written as (46).

APPENDIX F

RADIX r^4 DIF SDC/SDF FFT ALGORITHM DERIVATION

We rewrite (8) as

$$\mathbf{T}_N = \mathbf{P}_N^{(r^4)} \cdot [\mathbf{I}_{r^4} \otimes \mathbf{T}_{N/r^4}] \cdot \mathbf{D}_N^{(r^4)} \cdot [\mathbf{T}_{r^4} \otimes \mathbf{I}_{N/r^4}]. \quad (57)$$

We further decompose the term $\mathbf{T}_{r^4}$ into simpler butterflies using (8) and (39):

$$\begin{aligned} \mathbf{T}_{r^4} = & \mathbf{P}_{r^4}^{(r^2)} \cdot [\mathbf{I}_{r^2} \otimes \mathbf{P}_{r^2}^{(r)}] \cdot [\mathbf{I}_{r^3} \otimes \mathbf{T}_r] \cdot [\mathbf{I}_{r^2} \otimes \mathbf{D}_{r^2}^{(r)}] \\ & \cdot [\mathbf{I}_{r^2} \otimes \mathbf{T}_r \otimes \mathbf{I}_r] \cdot \mathbf{D}_{r^4}^{(r^2)} \cdot [\mathbf{P}_{r^2}^{(r)} \otimes \mathbf{I}_{r^2}] \\ & \cdot [\mathbf{I}_r \otimes \mathbf{T}_r \otimes \mathbf{I}_{r^2}] \cdot [\mathbf{D}_{r^2}^{(r)} \otimes \mathbf{I}_{r^2}] \cdot [\mathbf{T}_r \otimes \mathbf{I}_{r^3}]. \quad (58) \end{aligned}$$

Applying Lemmas 3, 4, 6, and 7, (58) can be rearranged as

$$\begin{aligned} \mathbf{T}_{r^4} = & [\mathbf{P}_{r^2}^{(r)} \otimes \mathbf{I}_{r^2}] \cdot [\mathbf{P}_{r^3}^{(r)} \otimes \mathbf{I}_r] \cdot \mathbf{P}_{r^4}^{(r)} \cdot [\mathbf{I}_{r^2} \otimes \mathbf{P}_{r^2}^{(r)}] \\ & \cdot [\mathbf{P}_{r^2}^{(r)} \otimes \mathbf{I}_{r^2}] \cdot [\mathbf{I}_{r^3} \otimes \mathbf{T}_r] \cdot [\mathbf{I}_{r^2} \otimes \mathbf{D}_{r^2}^{(r)}] \\ & \cdot [\mathbf{I}_{r^2} \otimes \mathbf{T}_r \otimes \mathbf{I}_r] \cdot [\mathbf{P}_{r^2}^{(r)} \otimes \mathbf{I}_{r^2}] \cdot \mathbf{D}_{r^4}^{(r^2)} \cdot [\mathbf{P}_{r^2}^{(r)} \otimes \mathbf{I}_{r^2}] \\ & \cdot [\mathbf{I}_r \otimes \mathbf{T}_r \otimes \mathbf{I}_{r^2}] \cdot [\mathbf{D}_{r^2}^{(r)} \otimes \mathbf{I}_{r^2}] \cdot [\mathbf{T}_r \otimes \mathbf{I}_{r^3}]. \quad (59) \end{aligned}$$

Replacing (59) in (57),

$$\begin{aligned} \mathbf{T}_N = & \mathbf{P}_N^{(r^4)} \cdot [\mathbf{I}_{r^4} \otimes \mathbf{T}_{N/r^4}] \cdot \mathbf{D}_N^{(r^4)} \cdot [\mathbf{P}_{r^2}^{(r)} \otimes \mathbf{I}_{N/r^2}] \\ & \cdot [\mathbf{P}_{r^3}^{(r)} \otimes \mathbf{I}_{N/r^3}] \cdot [\mathbf{P}_{r^4}^{(r)} \otimes \mathbf{I}_{N/r^4}] \\ & \cdot [\mathbf{I}_{r^2} \otimes \mathbf{P}_{r^2}^{(r)} \otimes \mathbf{I}_{N/r^4}] \cdot [\mathbf{P}_{r^2}^{(r)} \otimes \mathbf{I}_{N/r^2}] \\ & \cdot [\mathbf{I}_{r^3} \otimes \mathbf{T}_r \otimes \mathbf{I}_{N/r^4}] \cdot [\mathbf{I}_{r^2} \otimes \mathbf{D}_{r^2}^{(r)} \otimes \mathbf{I}_{N/r^4}] \\ & \cdot [\mathbf{I}_{r^2} \otimes \mathbf{T}_r \otimes \mathbf{I}_{N/r^3}] \cdot [\mathbf{P}_{r^2}^{(r)} \otimes \mathbf{I}_{N/r^2}] \\ & \cdot [\mathbf{D}_{r^4}^{(r^2)} \otimes \mathbf{I}_{N/r^4}] \cdot [\mathbf{P}_{r^2}^{(r)} \otimes \mathbf{I}_{N/r^2}] \\ & \cdot [\mathbf{I}_r \otimes \mathbf{T}_r \otimes \mathbf{I}_{N/r^2}] \cdot [\mathbf{D}_{r^2}^{(r)} \otimes \mathbf{I}_{N/r^2}] \\ & \cdot [\mathbf{T}_r \otimes \mathbf{I}_{N/r}]. \quad (60) \end{aligned}$$

Applying Lemmas 3, 4 and 6 to (60) and after some lengthy manipulations, we arrive to the basic recursive decomposition of the radix r^4 DIF SDC/SDF pipeline FFT algorithms:

$$\begin{aligned} \mathbf{T}_N = & \mathbf{P}_N^{(r^4)} \cdot [(\mathbf{P}_{r^4}^{(r)})^{-1} \otimes \mathbf{I}_{N/r^4}] \cdot [(\mathbf{P}_{r^3}^{(r)})^{-1} \otimes \mathbf{I}_{N/r^3}] \\ & \cdot [(\mathbf{P}_{r^2}^{(r)})^{-1} \otimes \mathbf{I}_{N/r^2}] \cdot [\mathbf{I}_{r^4} \otimes \mathbf{T}_{N/r^4}] \cdot \mathbf{E}_N^{(r)} \\ & \cdot [\mathbf{I}_{r^3} \otimes (\mathbf{P}_{N/r^3}^{(r)})^{-1}] \cdot [\mathbf{I}_{N/r} \otimes \mathbf{T}_r] \\ & \cdot [\mathbf{I}_{r^3} \otimes \mathbf{P}_{N/r^3}^{(r)}] \cdot [\mathbf{I}_{r^2} \otimes \mathbf{D}_{r^2}^{(r)} \otimes \mathbf{I}_{N/r^4}] \\ & \cdot [\mathbf{I}_{r^2} \otimes (\mathbf{P}_{N/r^2}^{(r)})^{-1}] \cdot [\mathbf{I}_{N/r} \otimes \mathbf{T}_r] \\ & \cdot [\mathbf{I}_{r^2} \otimes \mathbf{P}_{N/r^2}^{(r)}] \cdot [\mathbf{P}_{r^2}^{(r)} \otimes \mathbf{I}_{N/r^2}] \\ & \cdot [\mathbf{D}_{r^4}^{(r^2)} \otimes \mathbf{I}_{N/r^4}] \cdot [\mathbf{P}_{r^2}^{(r)} \otimes \mathbf{I}_{N/r^2}] \\ & \cdot [\mathbf{I}_r \otimes (\mathbf{P}_{N/r}^{(r)})^{-1}] \cdot [\mathbf{I}_{N/r} \otimes \mathbf{T}_r] \\ & \cdot [\mathbf{I}_r \otimes \mathbf{P}_{N/r}^{(r)}] \cdot [\mathbf{D}_{r^2}^{(r)} \otimes \mathbf{I}_{N/r^2}] \cdot (\mathbf{P}_N^{(r)})^{-1} \\ & \cdot [\mathbf{I}_{N/r} \otimes \mathbf{T}_r] \cdot \mathbf{P}_N^{(r)} \quad (61) \end{aligned}$$

where

$$\begin{aligned} \mathbf{E}_N^{(r)} = & [\mathbf{P}_{r^2}^{(r)} \otimes \mathbf{I}_{N/r^2}] \cdot [\mathbf{P}_{r^3}^{(r)} \otimes \mathbf{I}_{N/r^3}] \\ & \cdot [\mathbf{P}_{r^4}^{(r)} \otimes \mathbf{I}_{N/r^4}] \cdot \mathbf{D}_N^{(r^4)} \cdot [\mathbf{P}_{r^2}^{(r)} \otimes \mathbf{I}_{N/r^2}] \\ & \cdot [\mathbf{P}_{r^3}^{(r)} \otimes \mathbf{I}_{N/r^3}] \cdot [\mathbf{P}_{r^4}^{(r)} \otimes \mathbf{I}_{N/r^4}]. \quad (62) \end{aligned}$$

If we apply the decomposition in (61) $k-1$ times, we obtain

$$\mathbf{T}_N = \mathbf{C}^{(0)} \cdot \mathbf{C}^{(1)} \cdot \mathbf{C}^{(2)} \dots \mathbf{C}^{(k-1)} \cdot [\mathbf{I}_{r^{4k}} \otimes \mathbf{T}_{N/r^{4k}}] \cdot \mathbf{H}^{(k-1)} \cdot \mathbf{H}^{(k-2)} \cdot \mathbf{H}^{(k-3)} \dots \mathbf{H}^{(0)} \quad (63)$$

where

$$\begin{aligned} \mathbf{C}^{(i)} = & [\mathbf{I}_{r^{4i}} \otimes \mathbf{P}_{(N/r^{4i})}^{(r^4)}] \cdot [\mathbf{I}_{r^{4i}} \otimes \mathbf{P}_{r^4}^{(r^3)} \otimes \mathbf{I}_{N/r^{(4i+4)}}] \\ & \cdot [\mathbf{I}_{r^{3i}} \otimes \mathbf{P}_{r^3}^{(r^2)} \otimes \mathbf{I}_{N/r^{(3i+3)}}] \\ & \cdot [\mathbf{I}_{r^{2i}} \otimes \mathbf{P}_{r^2}^{(r)} \otimes \mathbf{I}_{N/r^{(2i+2)}}] \quad (64) \end{aligned}$$

$$\mathbf{T}_N = \begin{cases} \mathbf{Q}_N \cdot \mathbf{T}_N^{(p_3)}, & \text{for } N = r^{3n_3} \\ \mathbf{Q}_N \cdot [\mathbf{I}_{N/r} \otimes \mathbf{T}_r] \cdot \mathbf{T}_N^{(p_3)}, & \text{for } N = r^{3n_3+1} \\ \mathbf{Q}_N \cdot [\mathbf{I}_{N/r} \otimes \mathbf{T}_r] \cdot [\mathbf{I}_{N/r^2} \otimes \mathbf{D}_{r^2}^{(r)}] \\ \quad \cdot [\mathbf{I}_{N/r^2} \otimes (\mathbf{P}_{r^2}^{(r)})^{-1}] \cdot [\mathbf{I}_{N/r} \otimes \mathbf{T}_r] \cdot [\mathbf{I}_{N/r^2} \otimes \mathbf{P}_{r^2}^{(r)}] \cdot \mathbf{T}_N^{(p_3)}, & \text{for } N = r^{3n_3+2} \end{cases} \quad (56a)$$

$$\mathbf{T}_N^{(p_3)} = \prod_{m=0}^{n_3-1} \mathbf{H}^{(n_3-m-1)} \quad (56b)$$

$$\mathbf{Q}_N = \begin{cases} \prod_{i=0}^{n_3-1} \mathbf{C}^{(i)}, & \text{for } N = r^{3n_3}, N = r^{3n_3+1} \\ [\prod_{i=0}^{n_3-1} \mathbf{C}^{(i)}] \cdot [\mathbf{I}_{r^{3n_3}} \otimes \mathbf{P}_{r^2}^{(r)}], & \text{for } N = r^{3n_3+2} \end{cases} \quad (56c)$$

$$\mathbf{T}_N = \begin{cases} \mathbf{Q}_N \cdot \mathbf{T}_N^{(p_4)}, & \text{for } N = r^{4n_4} \\ \mathbf{Q}_N \cdot [\mathbf{I}_{N/r} \otimes \mathbf{T}_r] \cdot \mathbf{T}_N^{(p_4)}, & \text{for } N = r^{4n_4+1} \\ \mathbf{Q}_N \cdot [\mathbf{I}_{N/r} \otimes \mathbf{T}_r] \cdot [\mathbf{I}_{r^{4n_4}} \otimes \mathbf{D}_{r^2}^{(r)}] \cdot [\mathbf{I}_{r^{4n_4}} \otimes (\mathbf{P}_{N/r^{4n_4}}^{(r)})^{-1}] \\ \cdot [\mathbf{I}_{N/r} \otimes \mathbf{T}_r] \cdot [\mathbf{I}_{r^{4n_4}} \otimes \mathbf{P}_{N/r^{4n_4}}^{(r)}] \cdot \mathbf{T}_N^{(p_4)}, & \text{for } N = r^{4n_4+2} \\ \mathbf{Q}_N \cdot [\mathbf{I}_{N/r} \otimes \mathbf{T}_r] \cdot \mathbf{I}_{r^{4n_4}} \otimes ([\mathbf{P}_{r^2}^{(r)} \otimes \mathbf{I}_r] \cdot \mathbf{D}_{r^3}^{(r^2)} \cdot [\mathbf{P}_{r^2}^{(r)} \otimes \mathbf{I}_r]) \\ \cdot [\mathbf{I}_{r^{4n_4+1}} \otimes (\mathbf{P}_{r^2}^{(r)})^{-1}] \cdot [\mathbf{I}_{N/r} \otimes \mathbf{T}_r] \cdot [\mathbf{I}_{r^{4n_4+1}} \otimes \mathbf{P}_{r^2}^{(r)}] \\ \cdot [\mathbf{I}_{r^{4n_4}} \otimes \mathbf{D}_{r^2}^{(r)} \otimes \mathbf{I}_{N/r^{4n_4+2}}] \cdot [\mathbf{I}_{r^{4n_4}} \otimes (\mathbf{P}_{r^3}^{(r)})^{-1}] \\ \cdot [\mathbf{I}_{N/r} \otimes \mathbf{T}_r] \cdot [\mathbf{I}_{r^{4n_4}} \otimes \mathbf{P}_{r^3}^{(r)}] \cdot \mathbf{T}_N^{(p_4)}, & \text{for } N = r^{4n_4+3} \end{cases} \quad (66a)$$

$$\mathbf{T}_N^{(p_4)} = \prod_{m=0}^{n_4-1} \mathbf{H}^{(n_4-m-1)} \quad (66b)$$

$$\mathbf{Q}_N = \begin{cases} \prod_{i=0}^{n_4-1} \mathbf{C}^{(i)}, & \text{for } N = r^{4n_4}, N = r^{4n_4+1} \\ \prod_{i=0}^{n_4-1} \mathbf{C}^{(i)} \cdot [\mathbf{I}_{r^{4n_4}} \otimes \mathbf{P}_{r^2}^{(r)}], & \text{for } N = r^{4n_4+2} \\ \prod_{i=0}^{n_4-1} \mathbf{C}^{(i)} \cdot [\mathbf{I}_{r^{4n_4}} \otimes \mathbf{P}_{r^3}^{(r)}] \cdot [\mathbf{I}_{r^{4n_4+1}} \otimes \mathbf{P}_{r^2}^{(r)}], & \text{for } N = r^{4n_4+3} \end{cases} \quad (66c)$$

$$\begin{aligned} \mathbf{H}^{(i)} &= [\mathbf{I}_{r^{4i}} \otimes \mathbf{E}_{N/r^{4i}}^{(r)}] \cdot [\mathbf{I}_{r^{4i+3}} \otimes (\mathbf{P}_{N/r^{4i+3}}^{(r)})^{-1}] \\ &\cdot [\mathbf{I}_{N/r} \otimes \mathbf{T}_r] \cdot [\mathbf{I}_{r^{4i+3}} \otimes \mathbf{P}_{N/r^{4i+3}}^{(r)}] \\ &\cdot [\mathbf{I}_{r^{4i+2}} \otimes \mathbf{D}_{r^2}^{(r)} \otimes \mathbf{I}_{N/r^{4i+4}}] \\ &\cdot [\mathbf{I}_{r^{4i+2}} \otimes (\mathbf{P}_{N/r^{4i+2}}^{(r)})^{-1}] \cdot [\mathbf{I}_{N/r} \otimes \mathbf{T}_r] \\ &\cdot [\mathbf{I}_{r^{4i+2}} \otimes \mathbf{P}_{N/r^{4i+2}}^{(r)}] \\ &\cdot [\mathbf{I}_{r^{4i}} \otimes ([\mathbf{P}_{r^2}^{(r)} \otimes \mathbf{I}_{r^2}] \cdot \mathbf{D}_{r^4}^{(r^2)} \cdot [\mathbf{P}_{r^2}^{(r)} \otimes \mathbf{I}_{r^2}]) \\ &\otimes \mathbf{I}_{N/r^{4i+4}}] \cdot [\mathbf{I}_{r^{4i+1}} \otimes (\mathbf{P}_{N/r^{4i+1}}^{(r)})^{-1}] \\ &\cdot [\mathbf{I}_{N/r} \otimes \mathbf{T}_r] \cdot [\mathbf{I}_{r^{4i+1}} \otimes \mathbf{P}_{N/r^{4i+1}}^{(r)}] \\ &\cdot [\mathbf{I}_{r^{4i}} \otimes \mathbf{D}_{r^2}^{(r)} \otimes \mathbf{I}_{N/r^{4i+2}}] \cdot [\mathbf{I}_{r^{4i}} \otimes (\mathbf{P}_{N/r^{4i}}^{(r)})^{-1}] \\ &\cdot [\mathbf{I}_{N/r} \otimes \mathbf{T}_r] \cdot [\mathbf{I}_{r^{4i}} \otimes \mathbf{P}_{N/r^{4i}}^{(r)}]. \end{aligned} \quad (65)$$

When $N = r^{4k}$, then $\mathbf{T}_{N/r^{4k}} = \mathbf{T}_1 = 1$ and the iterations are over. When $N = r^{4k+1}$, then $\mathbf{T}_{N/r^{4k}} = \mathbf{T}_r$ and the iterations are over as well, because no further decomposition using radix r is possible. When $N = r^{4k+2}$, then $\mathbf{T}_{N/r^{4k}} = \mathbf{T}_{r^2}$ and it must be decomposed in terms of $\mathbf{T}_r$. When $N = r^{4k+3}$, then $\mathbf{T}_{N/r^{4k}} = \mathbf{T}_{r^3}$ and it must also be decomposed in terms of $\mathbf{T}_r$. Thus, we can write equations (66a)–(66c) shown at the top of the page, where $p_4 = r^{4(n_4-1)}$ and $\{n_4, p_4\} \in \mathbb{N}^*$. Using Lemma 7, (64) can be further manipulated so that (66c) is written as (46).

REFERENCES

- [1] L. Rabiner and B. Gold, *Theory and Application of Digital Signal Processing*. Englewood Cliffs, NJ: Prentice-Hall, 1975.
- [2] Y. Jung, H. Yoon, and J. Kim, "New efficient FFT algorithm and pipeline implementation results for OFDM/DMT applications," *IEEE Trans. Consum. Electron.*, vol. 49, pp. 14–20, Feb. 2005.
- [3] C. Cheng and K. K. Parhi, "High-throughput VLSI architecture for FFT computation," *IEEE Trans. Circuits Syst. II*, vol. 54, no. 10, pp. 863–867, Oct. 2007.
- [4] J. Lee, H. Lee, S. i. Cho, and S.-S. Choi, "A high-speed low-complexity two-parallel radix- 2^4 FFT/IFFT processor for UWB applications," in *Proc. IEEE Asian Solid-State Circuits Conf. (ASSCC)*, 2007, pp. 284–287.
- [5] M. Shin and H. Lee, "A high-speed four-parallel radix- 2^4 FFT/IFFT processor for UWB applications," in *Proc. IEEE Int. Symp. Circuits Systems (ISCAS)*, 2008, pp. 960–963.
- [6] G. Bi and E. Jones, "A pipelined FFT processor for word-sequential data," *IEEE Trans. Acoust., Speech, Signal Process.*, vol. 37, no. 12, pp. 1982–1985, Dec. 1989.
- [7] E. Wold and A. M. Despain, "Pipeline and parallel-pipeline FFT processors for VLSI implementations," *IEEE Trans. Comput.*, vol. C-33, pp. 414–426, May 1984.
- [8] S. He and M. Torkelson, "A new approach to pipeline FFT processor," in *Proc. IEEE 10th Int. Parallel Processing Symp.*, 1996, pp. 766–770.
- [9] S. He and M. Torkelson, "Designing pipeline FFT processor for OFDM (de)modulation," in *Proc. URSI Int. Symp. Signals, Systems, Electronics*, 1998, vol. 29, pp. 257–262.
- [10] J.-Y. Oh and M.-S. Lim, "New radix-2 to the 4-th power pipeline FFT processor," *IEICE Trans. Electron.*, vol. E88-C, pp. 1740–1746, Aug. 2005.
- [11] J.-Y. Oh and M.-S. Lim, "Area and power efficient pipeline FFT algorithm," in *Proc. IEEE Workshop Signal Processing Systems Design Implementation*, 2005, pp. 520–525.
- [12] J.-Y. Oh and M.-S. Lim, "Fast Fourier Transform algorithm for low-power and area-efficient implementation," *IEICE Trans. Commun.*, vol. E89-B, pp. 1425–1429, Apr. 2006.
- [13] A. Cortés, I. Vélez, A. Irizar, and J. F. Sevillano, "Area efficient IFFT/FFT core for MB-OFDM UWB," *IEE Electron. Lett.*, vol. 43, no. 11, pp. 649–650, May 2007.
- [14] A. Cortés, I. Vélez, I. Zalvide, A. Irizar, and J. F. Sevillano, "An FFT core for DVB-T/DVB-H receivers," *VLSI Design*, vol. 2008, no. ID 610420, 2008.
- [15] M. Pease, "An adaptation of the Fast Fourier Transform for parallel processing," *J. Assoc. Comput. Mach.*, vol. 15, no. 2, pp. 252–264, Apr. 1968.
- [16] M. J. Corinthios, "The design of a class of Fast Fourier Transform computers," *IEEE Trans. Comput.*, vol. 20, pp. 617–623, Jun. 1971.
- [17] H. Sloate, "Matrix representation for sorting and the FFT," *IEEE Trans. Circuits Syst.*, vol. CAS-21, pp. 109–116, 1974.
- [18] J. Granata, M. Conner, and R. Tolimieri, "Recursive fast algorithms and the role of the tensor product," *IEEE Trans. Signal Process.*, vol. 40, no. 12, pp. 2921–2930, Dec. 1992.
- [19] J. Johnson, R. W. Johnson, D. Rodriguez, and R. Tolimieri, "A methodology for designing, modifying and implementing Fourier transform algorithms on various architectures," *Circuits, Syst., Signal Process.*, vol. 9, pp. 449–500, 1990.
- [20] S. S. Agaian and D. Gevorkian, "Synthesis of a class of orthogonal transforms: Parallel SIMD-algorithms and specialized processors," *Pattern Recognit. Image Anal.*, vol. 2, no. 4, pp. 394–408, 1992.
- [21] R. Tolimieri, M. An, and C. Lu, *Algorithms for Discrete Fourier Transform and Convolution*. New York: Springer-Verlag, 1997.

- [22] D. Rodriguez, "Tensor product algebra as a tool for VLSI implementation of the discrete Fourier transform," in *Proc. IEEE Int. Conf. Acoustics, Speech, Signal Processing (ICASSP)*, 1991, pp. 1025–1028.
- [23] P. A. Milder, F. Franchetti, J. C. Hoe, and M. Püschel, "Formal datapath representation and manipulation for implementing DSP transforms," in *Proc. Design Automation Conf. (DAC)*, 2008, pp. 385–390.
- [24] J. E. Whelchel, J. P. O'Malley, W. J. Rinard, and J. F. McArthur, "The systolic phase rotation FFT—a new algorithm and parallel processor architecture," in *Proc. IEEE Int. Conf. Acoustics, Speech, Signal Processing (ICASSP)*, 1990, vol. 2, pp. 1021–1024.
- [25] S. F. Gorman and J. M. Wills, "Partial column FFT pipelines," *IEEE Trans. Circuits Syst. II*, vol. 42, no. 6, pp. 414–423, Jun. 1995.
- [26] Z. Qian, C. Lu, M. An, and R. Tolimieri, "Self-sorting in-place FFT algorithm with minimum working space," *IEEE Trans. Signal Process.*, vol. 42, no. 10, pp. 2835–2836, Oct. 1994.
- [27] J. Johnson, R. W. Johnson, D. A. Padua, and J. Xiong, "Searching for the best FFT formulas with the SPL compiler," *Lecture Notes Comput. Sci.*, vol. 2017, pp. 112–126, 2001.
- [28] M. Püschel, J. M. Moura, J. R. Johnson, D. Padua, M. M. Veloso, B. W. Singer, J. Xiong, F. Franchetti, A. Gacic, Y. Voronenko, K. Chen, R. W. Johnson, and N. Rizzolo, "Spiral: Code generation for DSP transforms," *Proc. IEEE*, vol. 93, pp. 232–273, 2005.
- [29] M.-H. Fan, C.-H. Huang, Y.-C. Chung, J.-S. Liu, and J.-Z. Lee, "A programming methodology for designing block recursive algorithms," *J. Inf. Sci. Eng.*, vol. 22, pp. 95–121, 2006.
- [30] J. W. Cooley and J. Tukey, "An algorithm for machine calculation of complex Fourier series," *Math. Comput.*, vol. 19, pp. 297–301, 1965.



Ainhoa Cortés received the M.Sc. degree in electrical, electronic, and control engineering and the Ph.D. degree, both from the University of Navarra, San Sebastián, Spain, in 2004 and 2007, respectively.

In February 2004, she joined CEIT, San Sebastián, Spain. She is also a Lecturer at the University of Navarra (TECNUN). Her research interests are in the area of digital communication system design and digital signal processing, specially FFT algorithms and architectures.



Igone Vélez (M'06) received the M.Sc. degree in electrical, electronic, and control engineering and the Ph.D. degree, both from the University of Navarra, San Sebastián, Spain, in 2000 and 2005, respectively.

Her work at CEIT, San Sebastián, Spain, involves the digital design of the physical layer of wireless communication systems, notably WLAN, RFID, and UWB. Her research interests include digital signal processing and hardware development. She is a Lecturer at TECNUN, University of Navarra, in the Electronic CAD Laboratory.



Juan F. Sevillano (M'08) received the degree in electrical, electronic, and control engineering and the Ph.D. degree, both from the University of Navarra, San Sebastián, Spain, in 1999 and 2004, respectively.

In 2000, he joined CEIT, San Sebastián, Spain, where he is involved in projects related to wireless communication. He is a Lecturer at TECNUN, University of Navarra, in the Electronic CAD Laboratory. His research interests are in the area of digital signal processing and digital hardware design.